Ch-2-Improving Deep Neural Networks: Hyperparameter Tuning, Regularization and Optimization

Lecture slides by

Dr. Sujata Pathak

IT, KJSCE

Hyperparameter tuning

- A machine learning model has two types of parameters.
  - The first type of parameters are the parameters that are learned through a machine learning model.
  - The second type of parameters are the hyper parameters that we pass to the machine learning model.
- Normally we randomly set the value for these hyper parameters and see what parameters result in best performance.
- However randomly selecting the parameters for the algorithm can be exhaustive.

Hyperparameter tuning

- Process of selecting the optimal values for a machine learning model's hyperparameters.
- Set before the actual training process begins and control aspects of the learning process itself.
- Effective tuning helps the model learn better patterns, avoid overfitting or underfitting and achieve higher accuracy on unseen data.

Hyperparameter tuning

- These hyperparameters might address model design questions such as:
  - What degree of polynomial features should I use for my linear model?
  - What should be the maximum depth allowed for my decision tree?
  - What should be the minimum number of samples required at a leaf node in my decision tree?
  - How many trees should I include in my random forest?
  - How many neurons should I have in my neural network layer?
  - How many layers should I have in my neural network?
  - What should I set my learning rate to for gradient descent?

Hyperparameter tuning

Model-based learning

Use the input data

$$\begin{bmatrix} x_{1,0} & x_{1,1} & \dots & x_{1,n} \\ x_{2,0} & x_{2,1} & \dots & x_{2,n} \\ \vdots & \vdots & \vdots & \vdots \\ x_{m,0} & x_{m,1} & \cdots & x_{m,n} \end{bmatrix} \text{ and } \begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_m \end{bmatrix}$$



To learn a set of parameters

$$\begin{bmatrix} \theta_0 & \theta_1 & \cdots & \theta_n \end{bmatrix}$$



Which yield a **generalized** function

$$f(x;\theta) = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$



Capable of predicting values or classes on new input data

$$f(x_i;\theta) = 39$$

$$f(x_j;\theta) = 1$$

Hyperparameter tuning

- In general, this process includes:
  - Define a model
  - Define the range of possible values for all
- hyperparameters
  - Define a method for sampling
- hyperparameter values
  - Define an evaluative criteria to judge the model
- Define a cross-validation method

Hyperparameter Optimization Algorithms-

Grid search

Random search

Bayesian Optimization

Hyperparameter tuning

- **Grid Search -**
  - Brute-force technique for hyperparameter tuning.
  - Trains the model using all possible combinations of specified hyperparameter values to find the best-performing setup.
  - Slow and uses a lot of computer power which makes it hard to use with big datasets or many settings.

Hyperparameter tuning

- **Grid Search**
- It works using below steps:
  - Create a grid of potential values for each hyperparameter.
  - Train the model for every combination in the grid.
  - Evaluate each model using cross-validation.
  - Select the combination that gives the highest score.

Hyperparameter tuning

- **Grid Search**
  - For example if we want to tune two hyperparameters C and Alpha for a Logistic Regression Classifier model with the following sets of values:
    C = [0.1, 0.2, 0.3, 0.4, 0.5]
    Alpha = [0.01, 0.1, 0.5, 1.0]

C	Alpha = 0.1	Alpha = 0.2	Alpha = 0.3	Alpha = 0.4
0.5	0.701	0.703	0.697	0.696
0.4	0.699	0.702	0.698	0.702
0.3	0.721	0.726	0.713	0.703
0.2	0.706	0.705	0.704	0.701
0.1	0.698	0.692	0.688	0.675

Hyperparameter tuning

- **Grid Search**
- **Example: Tuning Logistic Regression with GridSearchCV**
  - Generate sample data using make_classification.
  - Define a range of C values using logarithmic scale.
  - GridSearchCV tries all combinations from param_grid and uses 5-fold cross-validation.
  - It returns the best hyperparameter (C) and its corresponding validation score

Hyperparameter tuning

GridSearch

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV
import numpy as np
from sklearn.datasets import make_classification

X, y = make_classification(
    n_samples=1000, n_features=20, n_informative=10, n_classes=2,
random_state=42)

c_space = np.logspace(-5, 8, 15)
param_grid = {'C': c_space}

logreg = LogisticRegression()

logreg_cv = GridSearchCV(logreg, param_grid, cv=5)

logreg_cv.fit(X, y)

print("Tuned Logistic Regression Parameters: {}".format(logreg_cv.best_params_))
print("Best score is {}".format(logreg_cv.best_score_))
```

Tuned Logistic Regression Parameters: {'C': 0.006105402296585327}
Best score is 0.853

Hyperparameter tuning

- **Random Search**
  - As the name suggests RandomizedSearch picks random combinations of hyperparameters from the given ranges instead of checking every single combination like GridSearch.
  - In each iteration it tries a new random combination of hyperparameter values.
  - It records the model's performance for each combination.
  - After several attempts it selects the best-performing set.

Hyperparameter tuning

- **Random Search**
- **Example: Tuning Decision Tree with RandomizedSearchCV**
  - Define a range of values for each hyperparameter e.g, max_depth, min_samples_leaf etc.
  - Random combinations are picked and evaluated using 5-fold cross-validation.
  - The best combination and score are printed.

Hyperparameter tuning

```
import numpy as np
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=1000, n_features=20, n_informative=10,
n_classes=2, random_state=42)

from scipy.stats import randint
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import RandomizedSearchCV

param_dist = {
    "max_depth": [3, None],
    "max_features": randint(1, 9),
    "min_samples_leaf": randint(1, 9),
    "criterion": ["gini", "entropy"]
}

tree = DecisionTreeClassifier()
tree_cv = RandomizedSearchCV(tree, param_dist, cv=5)
tree_cv.fit(X, y)

print("Tuned Decision Tree Parameters: {}".format(tree_cv.best_params_))
print("Best score is {}".format(tree_cv.best_score_))
```

Tuned Decision Tree Parameters: {'criterion': 'entropy', 'max_depth': None, 'max_features': 6, 'min_samples_leaf': 6}

Best score is 0.8

Hyperparameter tuning

- **Bayesian Optimization**
  - Grid Search and Random Search can be inefficient because they blindly try many hyperparameter combinations, even if some are clearly not useful.
  - Bayesian Optimization treats hyperparameter tuning like a mathematical optimization problem and learns from past results to decide what to try next.

Hyperparameter tuning

- **Bayesian Optimization**
  - Build a probabilistic model (surrogate function) that predicts performance based on hyperparameters.
  - Update this model after each evaluation.
  - Use the model to choose the next best set to try.
  - Repeat until the optimal combination is found.
  - The surrogate function models:

$$P\big(score(y) \mid hyperparameters(x)\big)$$

Hyperparameter tuning

- **Bayesian Optimization**
  - Surrogate function models the relationship between hyperparameters x and the score y.
  - By updating this model iteratively with each new evaluation Bayesian optimization makes more informed decisions.
  - Common surrogate models used in Bayesian optimization include:
    - Gaussian Processes
    - Random Forest Regression
    - Tree-structured Parzen Estimators (TPE)

Hyperparameter tuning

Advantages of Hyperparameter tuning

- **Improved Model Performance**: Finding the optimal combination of hyperparameters can significantly boost model accuracy and robustness.
- **Reduced Overfitting and Underfitting**: Tuning helps to prevent both overfitting and underfitting resulting in a well-balanced model.
- **Enhanced Model Generalizability**: By selecting hyperparameters that optimize performance on validation data the model is more likely to generalize well to unseen data.
- **Optimized Resource Utilization**: With careful tuning resources such as computation time and memory can be used more efficiently avoiding unnecessary work.
- **Improved Model Interpretability**: Properly tuned hyperparameters can make the model simpler and easier to interpret.

Hyperparameter tuning

Challenges

- **Dealing with High-Dimensional Hyperparameter Spaces:** The larger the hyperparameter space the more combinations need to be explored. This makes the search process computationally expensive and time-consuming especially for complex models with many hyperparameters.
- **Incorporating Domain Knowledge: It** can help guide the hyperparameter search, narrowing down the search space and making the process more efficient. Using insights from the problem context can improve both the efficiency and effectiveness of tuning.
- **Developing Adaptive Hyperparameter Tuning Methods:** Dynamic adjustment of hyperparameters during training such as learning rate schedules or early stopping can lead to better model performance.

Batch Normalization

Why Normalization ?

- Neural networks may struggle when working with input features that have different value ranges, causing longer training times and convergence issues.
- Training models on unnormalized data leads to slower convergence due to gradient descent difficulties.
- The problem of exploding gradients arises, causing instability as high values propagate through network layers.
- Preprocessing techniques like normalization and standardization help overcome scaling issues.

Batch Normalization (BatchNorm)

- Deep learning technique that stabilizes and accelerates neural network training by normalizing the inputs to each layer, reducing internal covariate shift (changing input distributions).
- Works by standardizing layer inputs (mean=0, variance=1) within each mini-batch and then applying learnable scale (gamma) and shift (beta) parameters, allowing for higher learning rates, faster convergence, and improved generalization.

Batch Normalization (BatchNorm)

- **Need of Batch Normalization**
- Batch Normalization makes sure outputs of each layer stay steady as model learns.
- This helps model train faster and learn more effectively.
  - Solves the problem of internal covariate shift.
  - Makes training faster and more stable.
  - Allows use of higher learning rates.
  - Helps avoid vanishing or exploding gradients.
  - Can act like a regularizer sometimes reduce the need for dropout.

Why Batch Normalization ?

The three main hypothesis on why Batch Normalization actually work is as follows :

- Batch normalization (BN) reduces the "internal covariate shift" [1].
- BN influences the smoothness of the optimization landscape, and hence simplifies the training process. [2]
- BN removes the interdependency between layers during training [5].

What is Batch Normalization ?

- Batch normalization (BN) involves normalizing activation vectors in hidden layers using the mean and variance of the current batch's data.
- This normalization is implemented right before or after the application of the nonlinear function.
- Here n = the number of samples in a batch
- At each hidden layer, Batch Normalization transforms the signal as follow :

$$\mu = \frac{1}{n}\sum_i Z^{(i)} \quad \text{___(1)}$$

$$\sigma^2 = \frac{1}{n}\sum_i (Z^{(i)} - \mu)^2 \quad \text{___(2)}$$

$$Z^{(i)}_{norm} = \frac{Z^{(i)} - \mu}{\sqrt{\sigma^2 - \epsilon}} \quad \text{___(3)}$$

$$\hat{Z}^{(i)} = \gamma Z^{(i)}_{norm} + \beta \quad \text{___(4)}$$

What is Batch Normalization ?

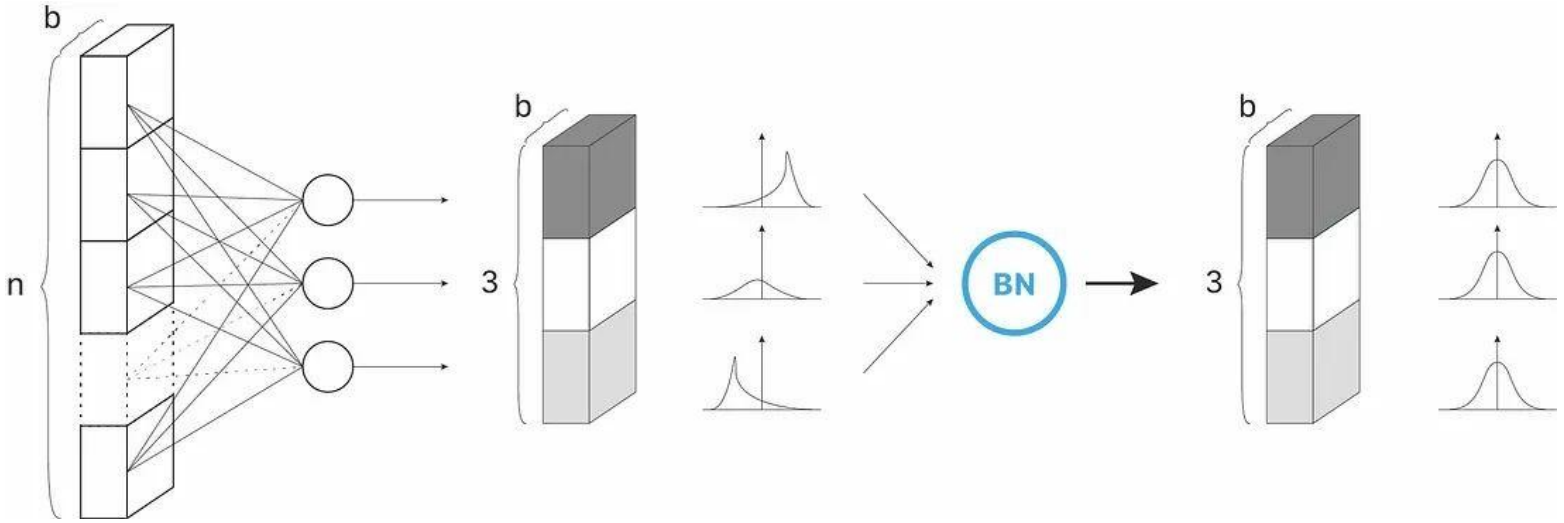
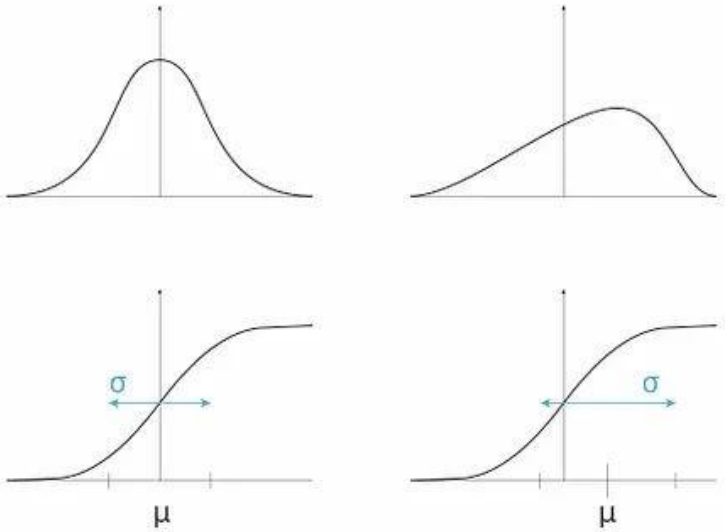


Fig 1: Batch Normalization first three steps [3]

- Example of a 3-neurons hidden layer, with a batch of size b.
- The BN layer first determines the mean μ and the variance σ^2 of the activation values across the batch, using equation (1) and (2) .
- It then normalizes the activation vector $Z^{(i)}$ using equation (3). Each neuron's output follows a standard normal distribution. [3]

What is Batch Normalization ?



$$\hat{Z}^{(i)} = \gamma Z^{(i)}_{norm} + \beta$$

Fig 2: Benefits of $\gamma$and βparameters. [3]

- It finally calculates the layer's output $\hat{Z}^{(i)}$ is calculated by applying a linear transformation with γ (scaling factor, adjust the standard deviation) and β(offset, adjust the bias), two trainable parameters as mentioned in equation(4).
- Modifying the distribution (on the top) allows us to use different regimes of the nonlinear functions (on the bottom).

Internal Covariate Shift

- Internal Covariate Shift (ICS) is the shifting of the model input distribution happening in the hidden layers of a deep neural network.
- Normalizing the input signal makes the points closer to each other in the feature space during training : it is now easier to find a well generalizing function.

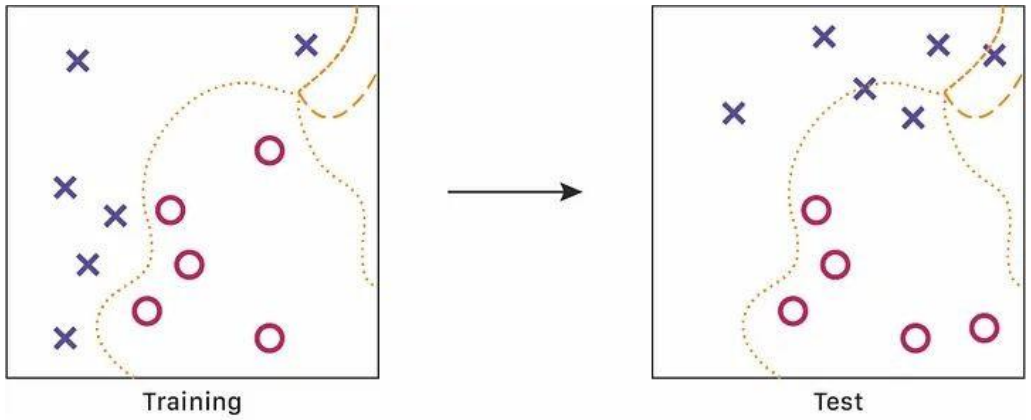


Fig 3: Plot of the extracted feature in the feature space. [3]

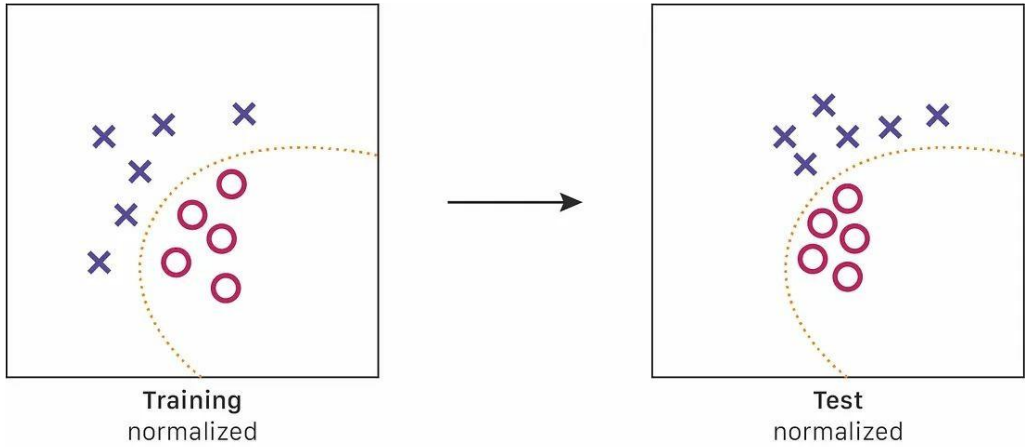


Fig 4: Normalizing the input signal makes the points closer to each other in the feature space during training : it is now easier to find a well generalizing function. [3]

Internal Covariate Shift

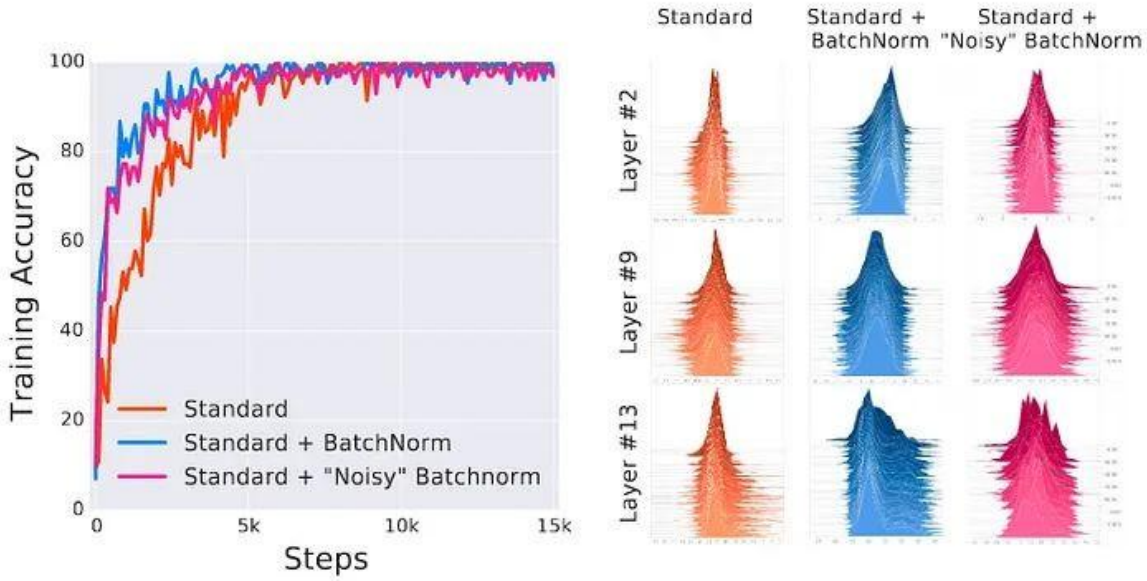


Fig 5: Networks with BN are trained faster than the standard one ; explicitly adding ICS_distrib on a regulated network does not deteriorate BN benefits. The trained model is a VGG networks (on CIFAR-10) . [2][3]

- The first network doesn't have any BN layer, while the second layer have BN layers.
- In the third one, they have explicitly added some ICS_distrib inside the hidden unit right before the activation (by adding random bias & variance).
- The noisy network is still trained faster than the standard one. Its reached performances are comparable to the ones obtained with a standard BN network.

Smoothness of the optimization landscape

- Normalization of the input signal inside hidden units , which makes the optimization landscape smoother.
- BN makes the optimization landscape smoother while preserving all of the minima of the normal landscape.

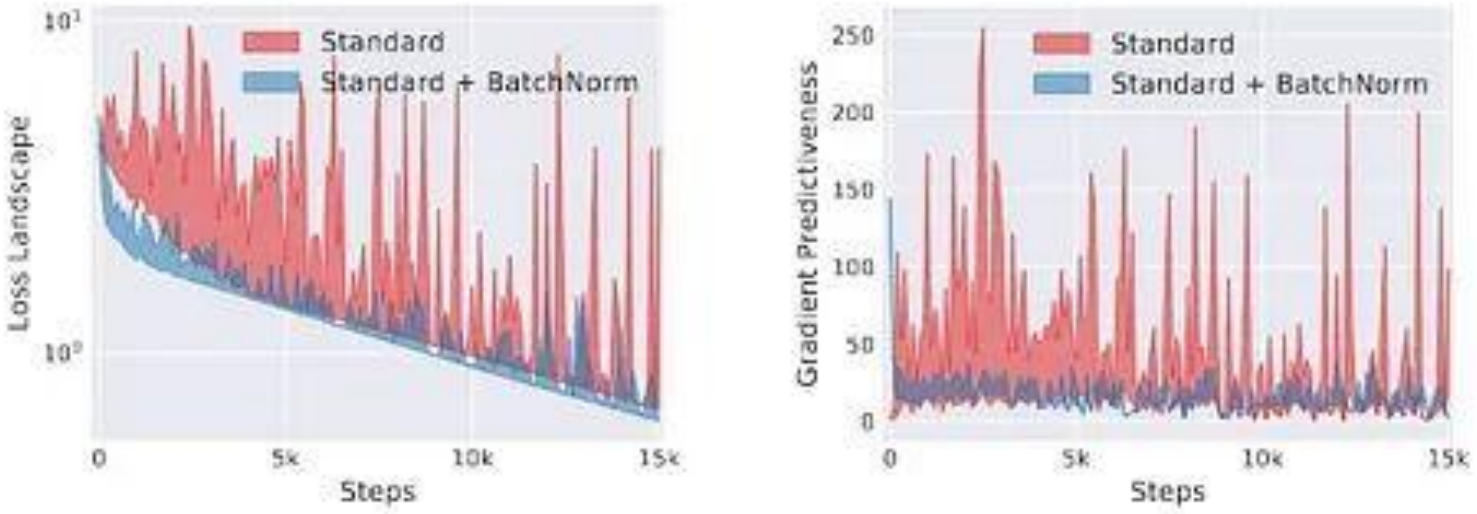


Fig 6 : Batch normalization impact on optimization landscape smoothening [2]

Interdependency between hidden layers

- From the fig.5 the gradient from the networks output:
  *grad(a) = grad(b) *grad(c) *grad(d) *grad(e)*
- This interdependency is problematic for the input stability.
- Batch Normalization also has a beneficial effect on the gradient flow through the network, by reducing the dependence of gradients on the scale of the parameters or of their initial values. This allows for use of much higher learning rates without the risk of divergence.

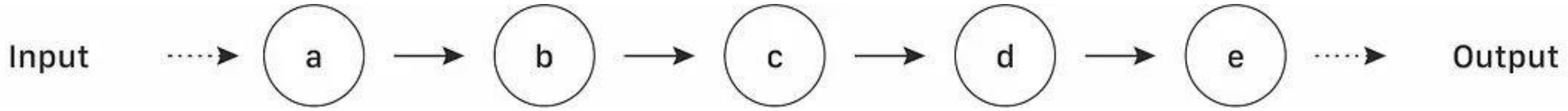


Fig 3: A simple DNN , which consists of linear transformation and (a), (b), (c), (d) and (e) are the sequential layers of the network. [3] [5]

Batch Normalization : Results

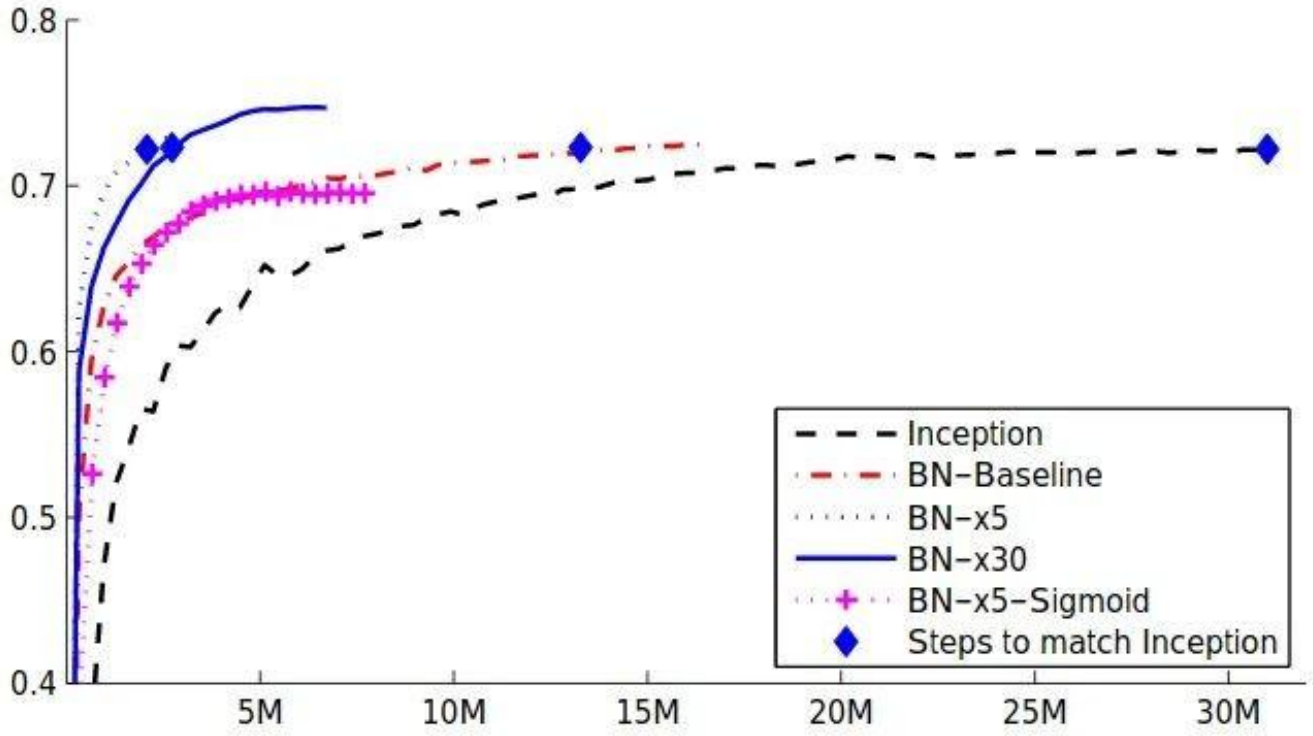


Fig : Batch normalization impact on training (ImageNet) : “Inception” : original network ; “BN-Baseline” : same as Inception with BN, same learning rate (LR) ; “BN-x5” : same as Inception with BN, LR x5 ; “BX-x30” : same as Inception with BN, LR x30 ; “BN-x5-Sigmoid” : same as Inception with BN, LR x5 and sigmoid instead of ReLU [1]

Limitations of Batch Normalization

- When using batch normalization, the sample mean and standard deviation from the current mini-batch may not be representative, particularly with small batch sizes. This can hinder the network from learning effectively. [9] [10]

- Batch normalization is less suitable for sequence models, especially when handling sequences of varying lengths and smaller batch sizes, as it relies on batch statistics for normalization. [6]

Batch Normalization

Batch Norm Regularization Effect

- Each hidden units are multiplied by a random value at each step of training

→ Add noises to training process

→ Force layers to learn harder to be robust a lot of variation of inputs

→ A form of data augmentation

Batch Normalization (BatchNorm)

- **Fundamentals of Batch Normalization**
  - **Step 1: Compute the Mean and Variance of Mini-Batches**
    - For mini-batch of activations $x_1, x_2, \ldots, x_m$,the mean μ_B and variance σ_B^2 of the mini-batch are computed.
  - **Step 2: Normalization**
    - Normalize each activation $x_i$is using the computed mean and variance of the mini-batch.
    - Add a small constant ϵ to the denominator for numerical stability, particularly to prevent division by zero.

$$\widehat{x_i} = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

Batch Normalization (BatchNorm)

- **Fundamentals of Batch Normalization**
  - **Step 3: Scale and Shift the Normalized Activations**
    - Scale normalized activations x^i by a learnable parameter γ and shifted by another learnable parameter β.
    - These parameters allow the model to learn the optimal scaling and shifting of the normalized activations giving the network additional flexibility.

$$y_i = \gamma \hat{x_i} + \beta$$

Batch Norm Algorithm

- Batch Norm tries to fix the means and variances of layer inputs
  - Reduce Internal Covariate Shift
- Run over batch axis

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: γ, β

Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma \widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

Batch Normalization (BatchNorm)

- **Batch Normalization in TensorFlow**

```
import tensorflow as tf

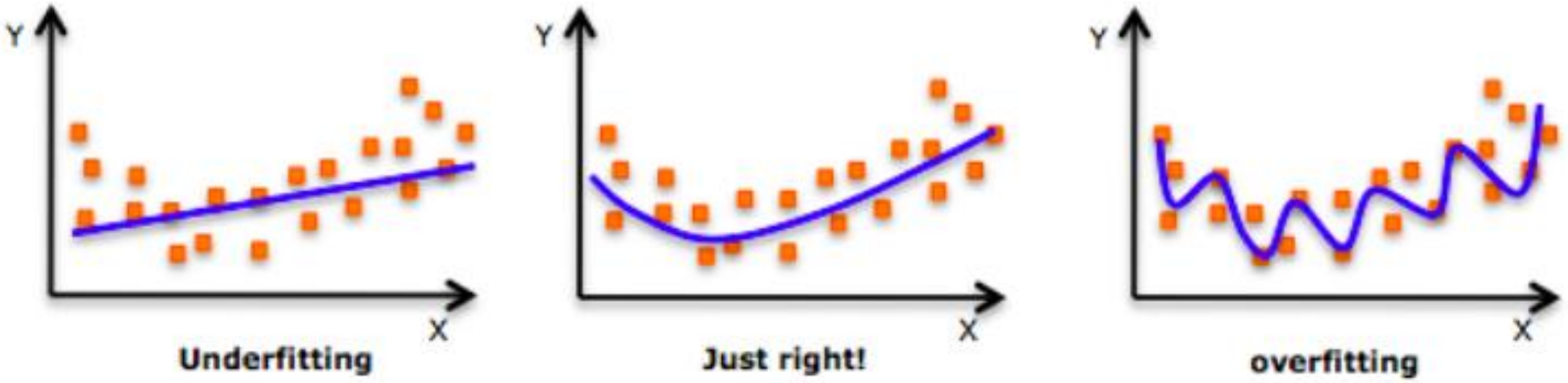
# Define a simple model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, input_shape=(784,)),
    # Add Batch Normalization layer
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Activation('relu'),
    tf.keras.layers.Dense(10),
    tf.keras.layers.Activation('softmax')
])

# Compile the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Train the model
model.fit(x_train, y_train, epochs=5, batch_size=32)
```

Regularization Techniques

What is regularization?



Overfitting causes the neural network model to perform very well during training, but the performance gets much worse during inference time when faced with brand new data.

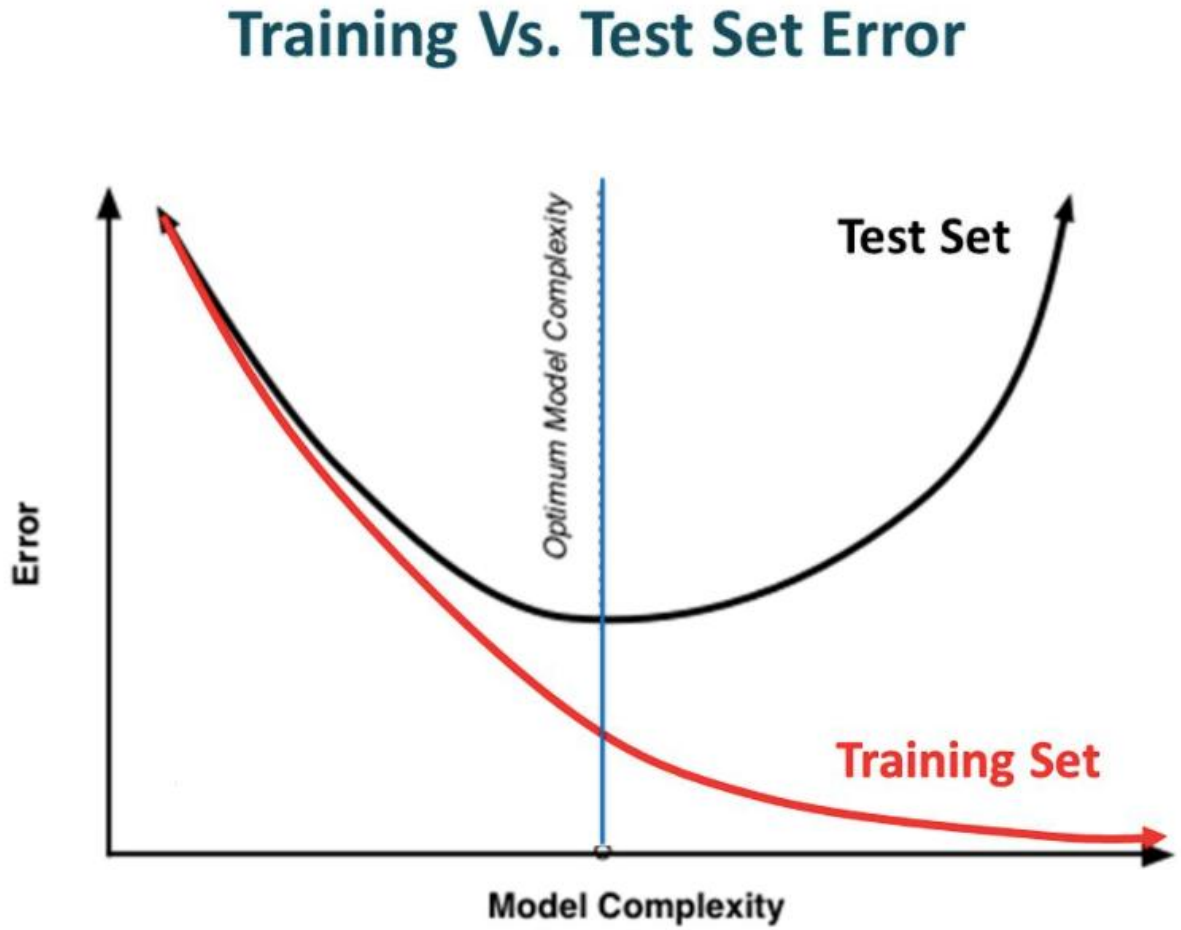
Counter underfit

- What causes underfit?
  - Model capacity is too small to fit the training dataset as well as generalize to new dataset.
  - High bias, low variance
- Solution
  - Increase the capacity of the model
  - Examples:
    - Increase number of layers, neurons in each layer, etc.
- Result:
  - Lower Bias
  - ***Underfit → Good Fit?***

Counter overfit

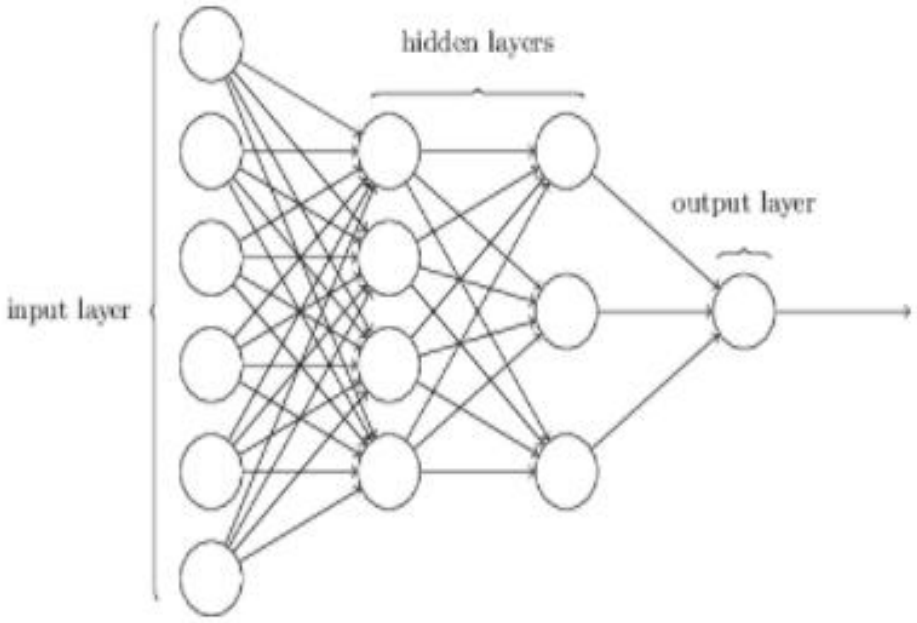
- What cause overfit?
  - Model capacity is so big that it adapts too well to training samples → Unable to generalize well to new, unseen samples
  - Low bias, high variance
- Solution
  - Regularization
- **But How?**

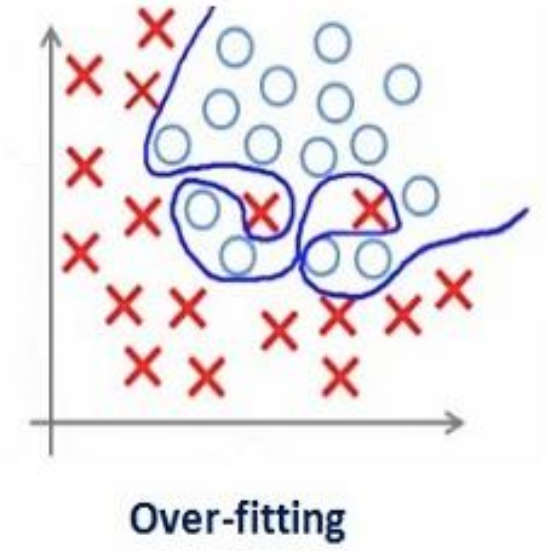
What is regularization?



- Regularization is a technique which makes slight modifications to the learning algorithm such that the model generalizes better.
- Regularization refers to a set of different techniques that lower the complexity of a neural network model during training and thus prevent overfitting.
- This in turn improves the model's performance on the unseen data as well.

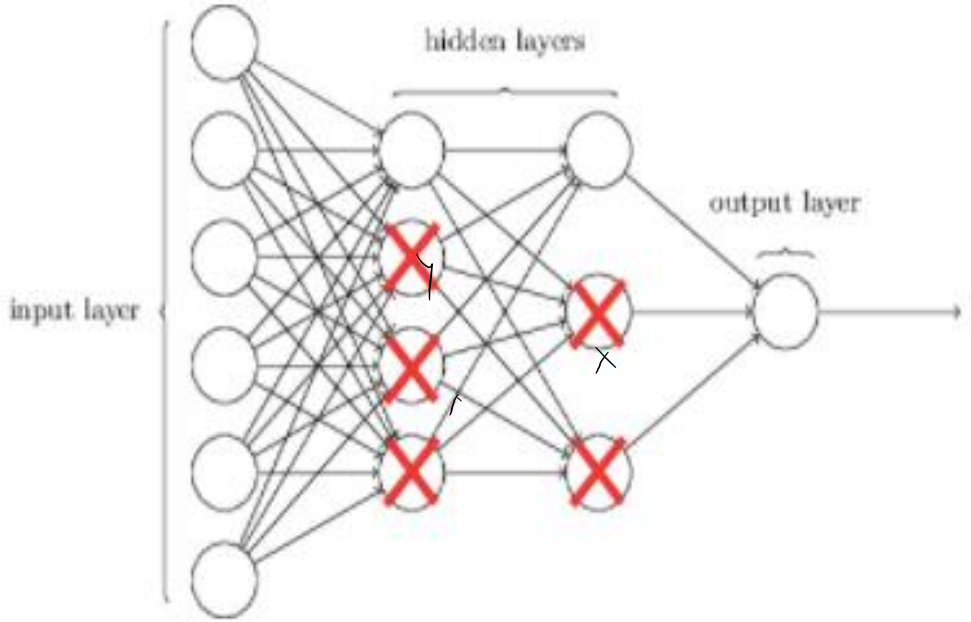
How does Regularization help reduce Overfitting?

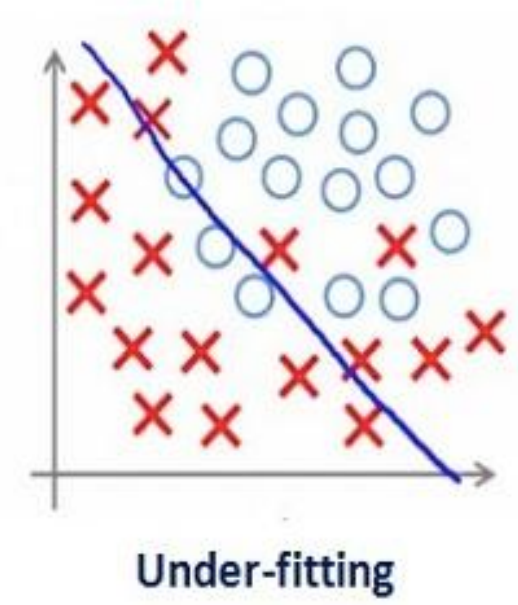




- Regularization penalizes the coefficients.
- In deep learning, it actually penalizes the weight matrices of the nodes.

How does Regularization help reduce Overfitting?





- Assume that our regularization coefficient is so high that some of the weight matrices are nearly equal to zero.
- This will result in a much simpler linear network and slight underfitting of the training data
- Optimize the value of regularization coefficient in order to obtain a well-fitted model

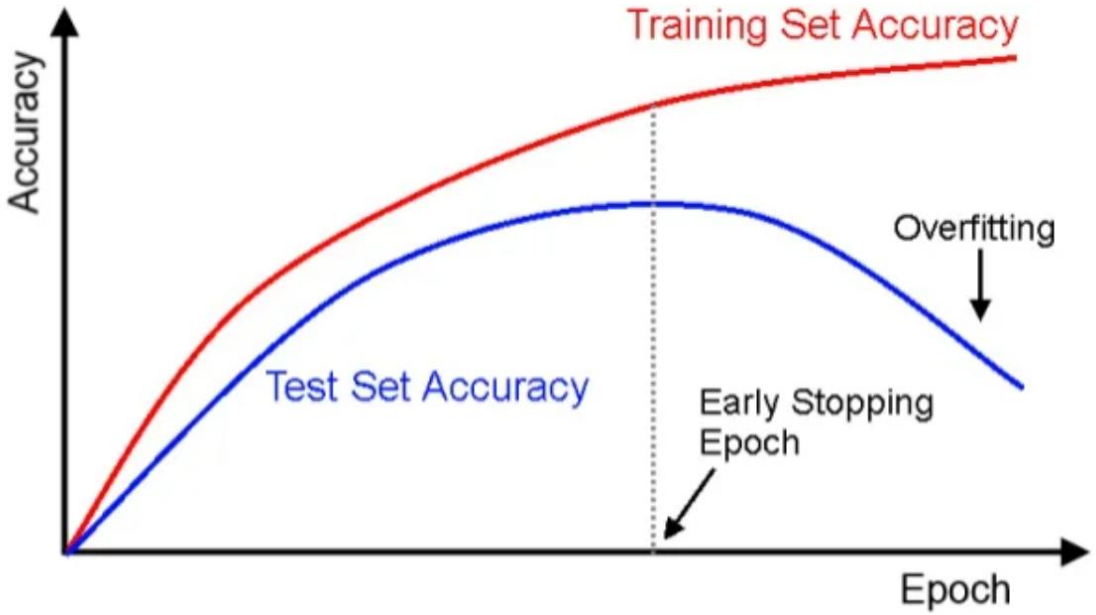
Different Regularization techniques in Deep Learning

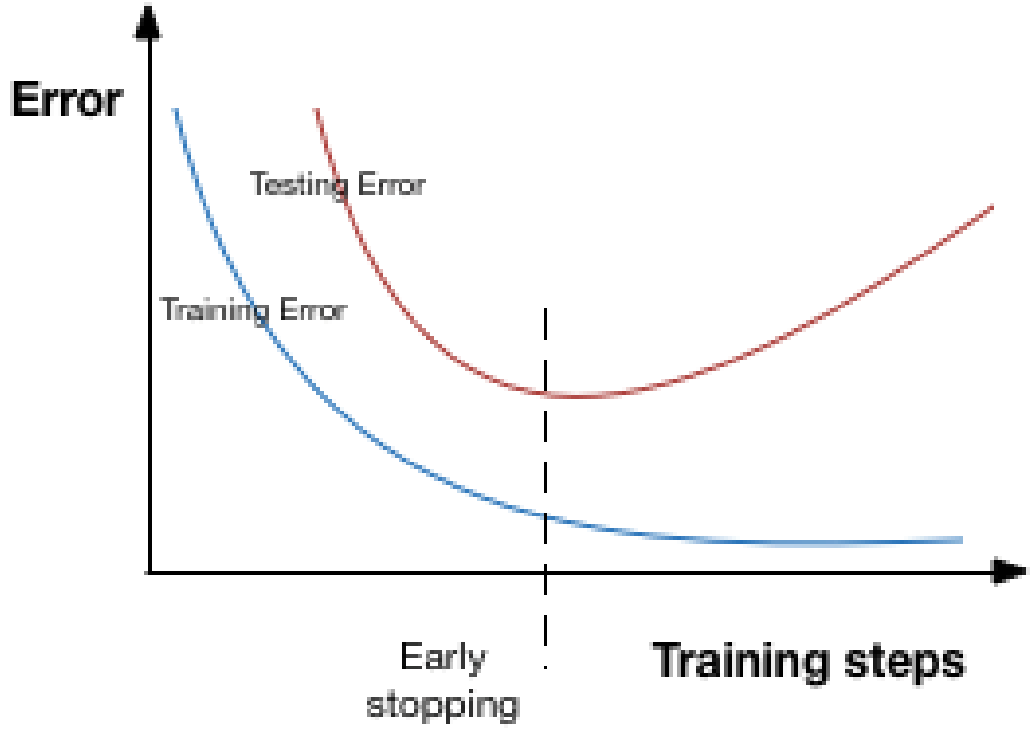
- Early stopping
- Batch Normalization
- L2 and L1 regularization
- Dropout
- Data augmentation

Early stopping

- There is point during training a large neural net when the model will stop generalizing and only focus on learning the statistical noise in the training dataset.
- Solution
  - Stop whenever generalization errors increases

Early stopping





Early stopping

- Kind of cross-validation strategy where we keep one part of the training set as the validation set.
- When we see that the performance on the validation set is getting worse, we immediately stop the training on the model.
- This is known as early stopping.
- In keras, apply early stopping using the callback function

```
from keras.callbacks import EarlyStopping

EarlyStopping(monitor='val_err', patience=5)
```

Early stopping

- **Key Parameters in Early Stopping**
  - **Patience:** The number of epochs to wait for validation improvement before stopping, typically between 5 to 10 epochs.
  - **Monitor Metric**: The metric to track during training, often validation loss or validation accuracy.
  - **Restore Best Weights**: After stopping, the model reverts to the weights from the epoch with the best validation performance.

Early stopping

- **Benefits of Early Stopping**
  - **Reduces Overfitting:** By stopping training when overfitting starts, early stopping improves generalization to unseen data.
  - **Saves Computational Resources:** Training for too many epochs can be time-consuming and costly. It stops training once further improvement is unlikely, saving time and resources.
  - **Improves Model Efficiency:** It leads to better-performing models in fewer epochs making the process more efficient.
  - **Simple to Implement:** It is a straightforward technique that can be easily applied using built-in tools in most machine learning libraries.

L1 Regularization

- Also knows as Lasso (Least Absolute Shrinkage and Selection Operator) regression
- Simply use another regularization term Ω.
- Adds the absolute value of magnitude of the coefficient as a penalty term to the loss function(L).

$$\Omega(\boldsymbol{\theta}) = ||\boldsymbol{w}||_1 = \sum_i |w_i|$$

- Multiply the regularization term by alpha and add the entire thing to the loss function.

$$\hat{\mathcal{L}}(W) = \alpha||W||_1 + \mathcal{L}(W)$$

Loss function during L1 Regularization

L1 Regularization

$$\text{Cost} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^{m} |w_i|$$

- Where
  - m: Number of Features
  - n: Number of Examples
  - y_i :Actual Target Value
  - $\hat{y}_i$:Predicted Target Value

L1 Regularization

```
from sklearn.linear_model import Lasso
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_regression
from sklearn.metrics import mean_squared_error

X, y = make_regression(n_samples=100, n_features=5, noise=0.1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)

y_pred = lasso.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
print(f"Mean Squared Error: {mse}")

print("Coefficients:", lasso.coef_)
```

L2 Regularization

- Most common type of all regularization techniques
- Also commonly known as weight decay or ridge regression or Tikhonov regularization.
- It adds the squared magnitude of the coefficient as a penalty term to the loss function(L).
- During the L2 regularization the loss function of the neural network as extended by a so-called regularization term, which is called here Ω

$$\Omega(\boldsymbol{\theta}) = \frac{1}{2}\|\boldsymbol{w}\|_2^2$$

L2 Regularization

- The regularization term Ω is defined as –
  - Euclidean Norm (or L2 norm) of the weight matrices, which is the sum of over all squared weight values of a weight matrix.
- The regularization term is weighted by the scalar alpha divided by two and added to the regular loss function that is chosen for the current task.
- This leads to a new expression for the loss function:

$$\hat{\mathcal{L}}(W) = \frac{\alpha}{2}||W||_2^2 + \mathcal{L}(W) = \frac{\alpha}{2}\sum_i \sum_j w_{ij}^2 + \mathcal{L}(W)$$

Regularization loss during L2 regularization

L2 Regularization

- Alpha is sometimes called as the regularization rate.
- Additional hyperparameter we introduce into the neural network and determines how much we regularize our model.
- Compute the gradient of the new loss function and put the gradient into the update rule for the weights:

$$\nabla_W \hat{\mathcal{L}}(W) = \alpha W + \nabla_W \mathcal{L}(W)$$

$$W_{new} = W_{old} - \epsilon(\alpha W_{old} + \nabla_W \mathcal{L}(W_{old}))$$

Gradient Descent during L2 Regularization.

L2 Regularization

- Reformulate the update rule leading to the expression similar to update rule for the weights during regular gradient descent:

$$W_{new} = (1 - \epsilon\alpha)W_{old} - \epsilon\nabla_W \mathcal{L}(W_{old})$$

- The only difference is that by adding the regularization term we introduce an additional subtraction from the current weights (first term in the equation).
- In other words independent of the gradient of the loss function we are making our weights a little bit smaller each time an update is performed.

L2 Regularization

```
from sklearn.linear_model import Ridge
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

X, y = make_regression(n_samples=100, n_features=5, noise=0.1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)
y_pred = ridge.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
print("Coefficients:", ridge.coef_)
```

L1/L2 Regularization

- L2 adds "***squared magnitude***" of coefficient as penalty term to the loss function.

$$Loss = Loss + \lambda \sum \beta^2$$

- L1 adds "***absolute value of magnitude***" of coefficient as penalty term to the loss function.

$$Loss = Loss + \lambda \sum |\beta|$$

- Weight Penalties → Smaller Weights → Simpler Model → Less Overfit

L1/L2 comparison

- Robustness
- Sparsity

Robustness (Against Outliers)

- L1>L2
- The loss of outliers increase
  - Exponentially in L2
  - Linearly in L1
- L2 pays more efforts to deal with outliers → Less Robust

L1/L2 comparison

Sparsity

- L1>L2
- L1 zeros out coefficients, which leads to a sparse model
- L1 can be used for feature (coefficients) selection
  - Unimportant ones have zero coefficients
- L2 will produce small values for almost all coefficients
- E.g: When applying L1/L2 to a layer with 4 weights, the results might look like
  - L1: 0.8, 0, 1, 0
  - L2: 0.3,0.1,0.3, 0.2

What does Regularization achieve?

- Performing L2 regularization encourages the weight values towards zero (but not exactly zero)
- Performing L1 regularization encourages the weight values to be zero

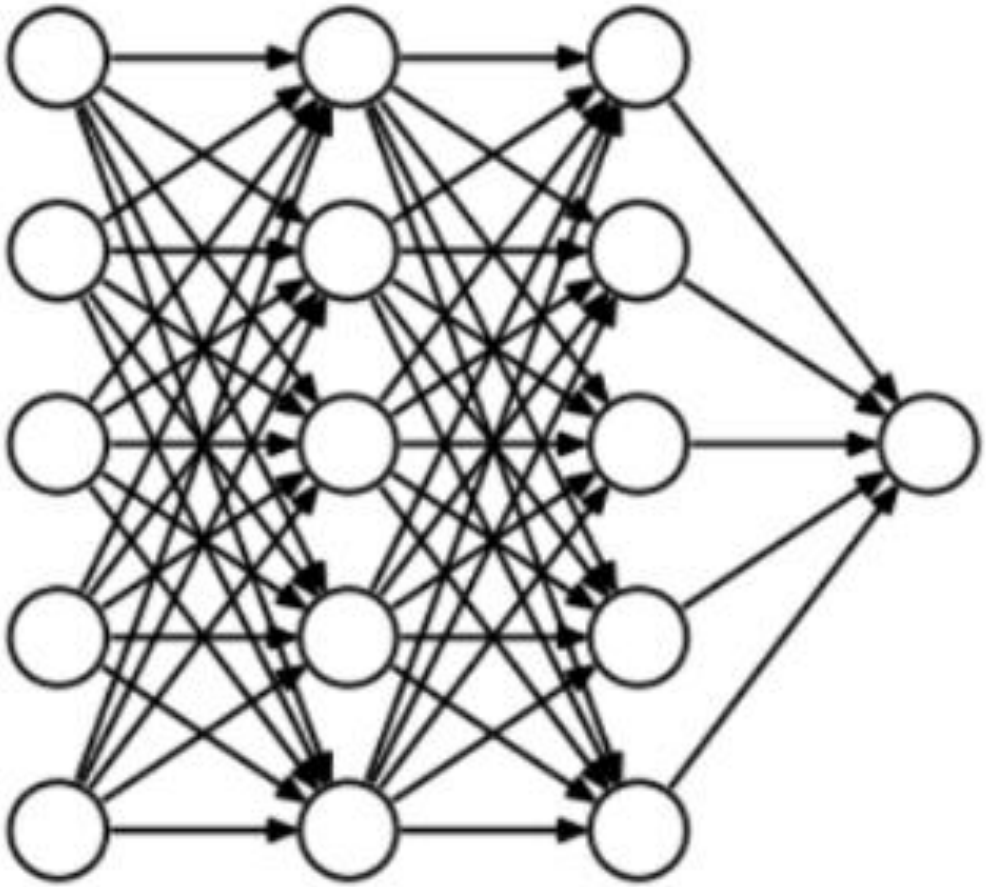
Selecting regularization term **α.**

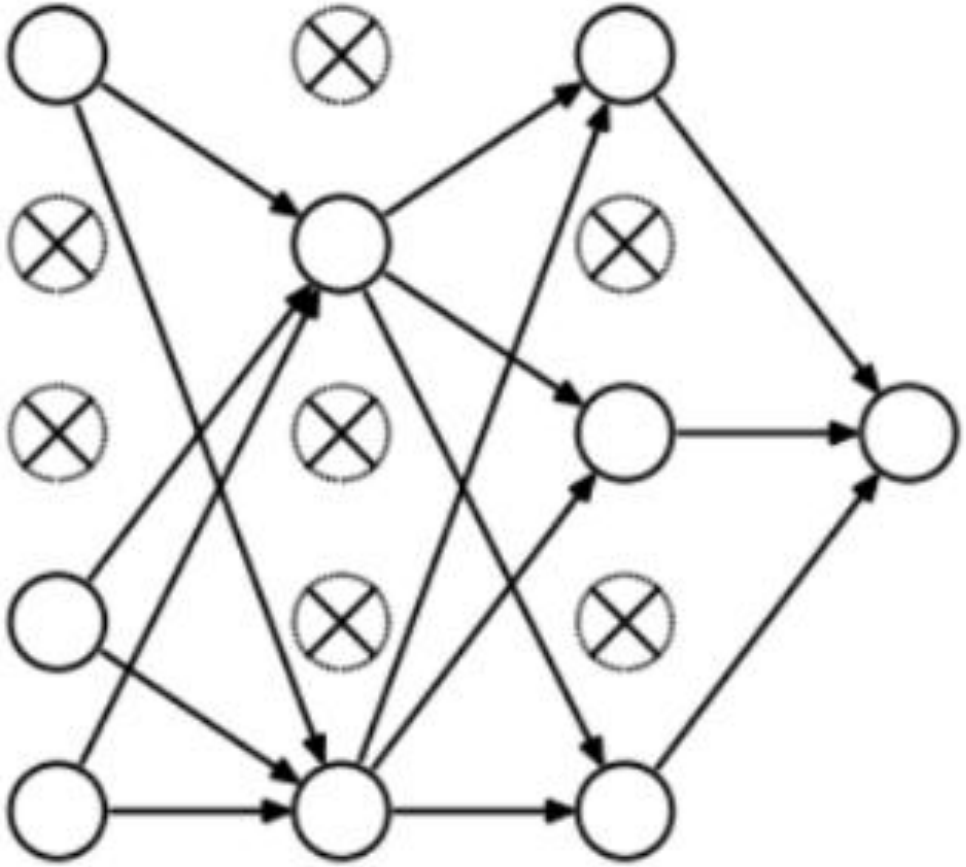
- If alpha value is too high, model will be simple, but you run the risk of underfitting your data. Your model won't learn enough about the training data to make useful predictions.
- If alpha value is too low, model will be more complex, and you run the risk of overfitting your data. Your model will learn too much about the particularities of the training data, and won't be able to generalize to new data.

Dropout

- Produces very good results
- Most frequently used regularization technique in the field of deep learning.
- During training with some probability P a neuron of the neural network gets turned off.
- Each iteration has a different set of nodes and this results in a different set of outputs.
- **It can also be thought of as an ensemble technique in machine learning.**

Dropout





Dropout

- Probability of choosing how many nodes should be dropped is the hyperparameter of the dropout function.
- Dropout can be applied to both the hidden layers as well as the input layers.
- Dropout is usually preferred when we have a large neural network structure in order to introduce more randomness.
- During dropout, some neurons get deactivated with a random probability P → Neural network becomes less complex → less overfitting

Dropout

- In keras, implement dropout using the keras core layer.

```
from keras.layers.core import Dropout

model = Sequential([
 Dense(output_dim=hidden1_num_units, input_dim=input_num_units, activation='relu')
 Dropout(0.25),

Dense(output_dim=output_num_units, input_dim=hidden5_num_units, activation='softma
 ])
```

Data Augmentation

- The simplest way to reduce overfitting is to increase the size of the training data.
- In machine learning- labeled data too costly.
- rotating the image, flipping, scaling, shifting, etc



- This technique is known as data augmentation.
- Provides a big leap in improving the accuracy of the model.
- Can be considered as a mandatory trick in order to improve our predictions.

Data Augmentation

- In *keras*, perform all of these transformations using ImageDataGenerator.

```
from keras.preprocessing.image import ImageDataGenerator
datagen = ImageDataGenerator(horizontal flip=True)
datagen.fit(train)
```

LAYER NORMALIZATION

Why Layer Normalization ?

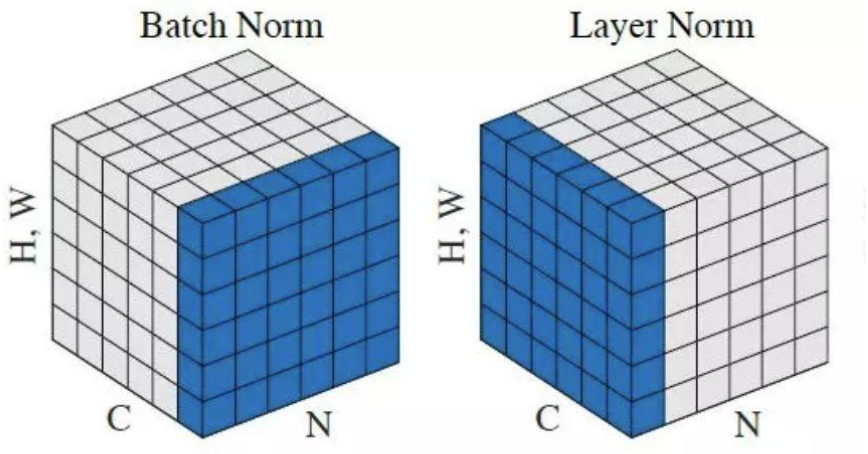


Fig 1: A feature map tensor, with N as the batch axis, C as the channel axis, and (H, W) as the spatial axes. The pixels in blue are normalized by the same mean and variance.

- Layer normalization is independent of the batch size, so it can be applied to batches with smaller sizes as well.
- Layer normalization has been shown to improve both the training time and the generalization performance of several existing RNN models.

What is Layer Normalization ?

- In Layer Normalization, all neurons within a layer share the same distribution across all input features.
- Here n = dimension of the input feature vector.
- The normalization is done along the length of the d dimensional vector and not the batch size.

$$\mu = \frac{1}{n}\sum_i Z^{(i)} \quad \text{___(1)}$$

$$\sigma^2 = \frac{1}{n}\sum_i (Z^{(i)} - \mu)^2 \quad \text{___(2)}$$

$$Z^{(i)}_{norm} = \frac{Z^{(i)} - \mu}{\sqrt{\sigma^2 - \epsilon}} \quad \text{___(3)}$$

$$\hat{Z}^{(i)} = \gamma Z^{(i)}_{norm} + \beta \quad \text{___(4)}$$

How Layer Normalization different from Batch Normalization ?

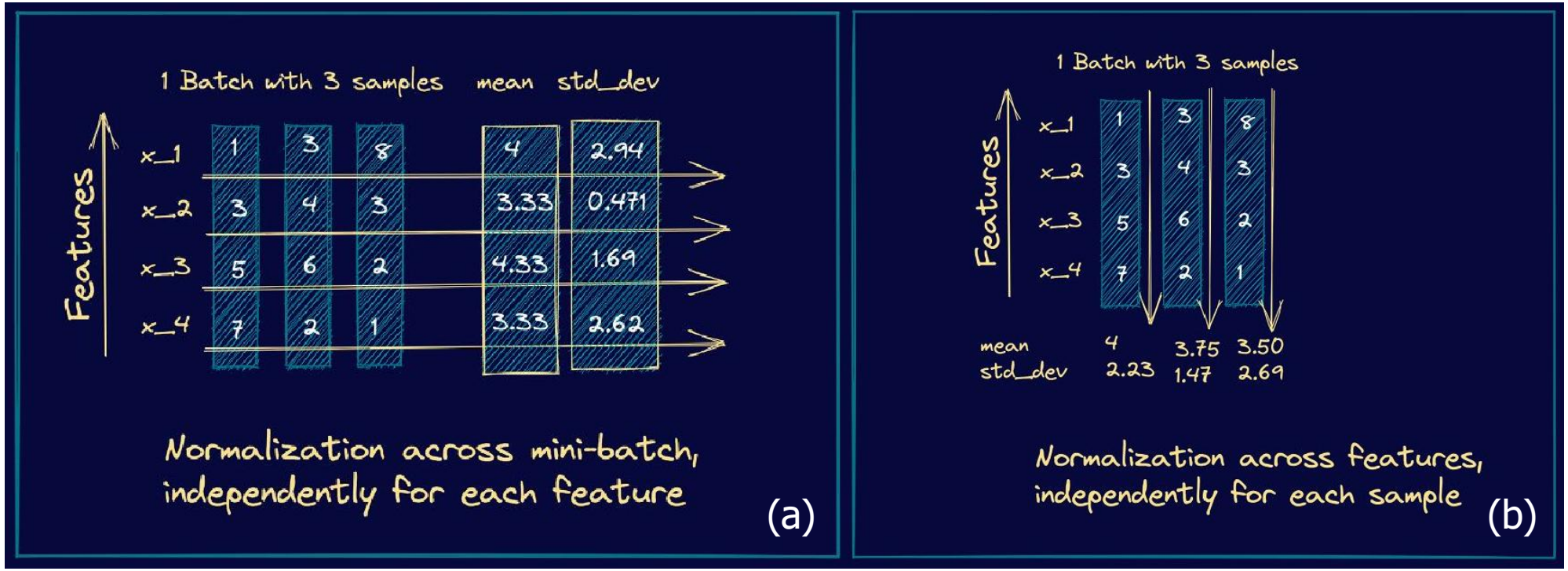


Fig 1: a) How Batch Normalization Works - An Example ,b) How Layer Normalization Works - An Example. [4]

Some other Normalizations

Instance Normalization

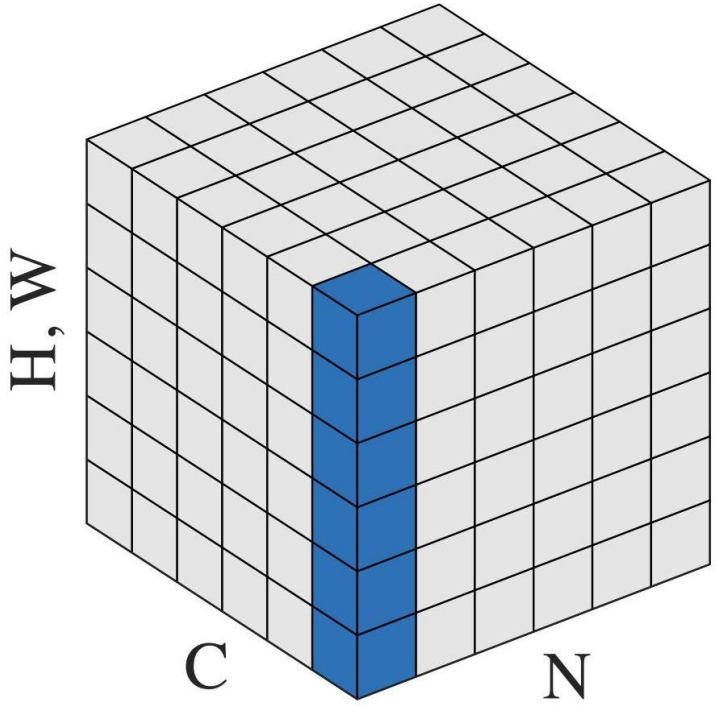


Fig 1: A feature map tensor, with N as the batch axis, C as the channel axis, and (H, W) as the spatial axes. The pixels in blue are normalized by the same mean and variance, computed across a single instance.

- Instance Normalization operates at the instance level, independently normalizing each sample (or instance) in a batch.
- It is frequently used in tasks related to style transfer, image-to-image translation, and generative models.
- It helps maintain the statistical properties of each instance, making it suitable for preserving the style or characteristics of individual images in these tasks.
- Instance Normalization is less dependent on batch statistics, which can be advantageous in scenarios where the batch size varies or is very small.

Group Normalization

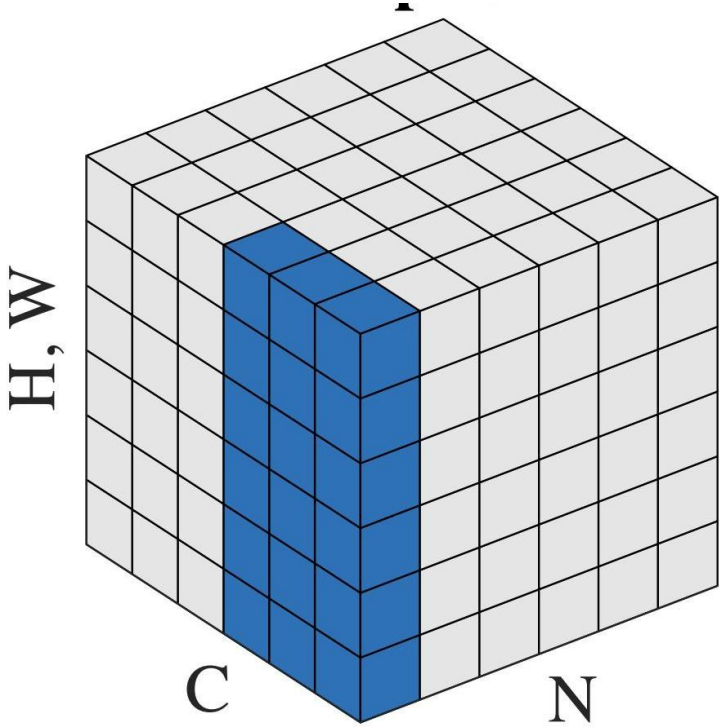


Fig 1: A feature map tensor, with N as the batch axis, C as the channel axis, and (H, W) as the spatial axes. The pixels in blue are normalized by the same mean and variance, computed across a multiple instances.

- Group Normalization divides the channels into groups and normalizes each group independently.
- It is useful when working with smaller batch sizes, as it reduces the dependence on batch statistics.
- Group Normalization is commonly applied in various computer vision tasks and is well-suited for convolutional neural networks (CNNs).

Comparison

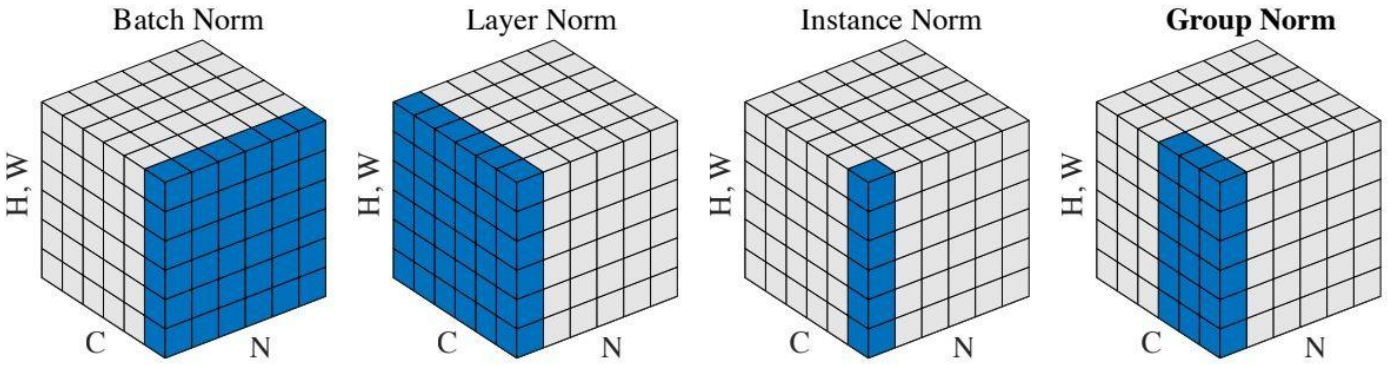
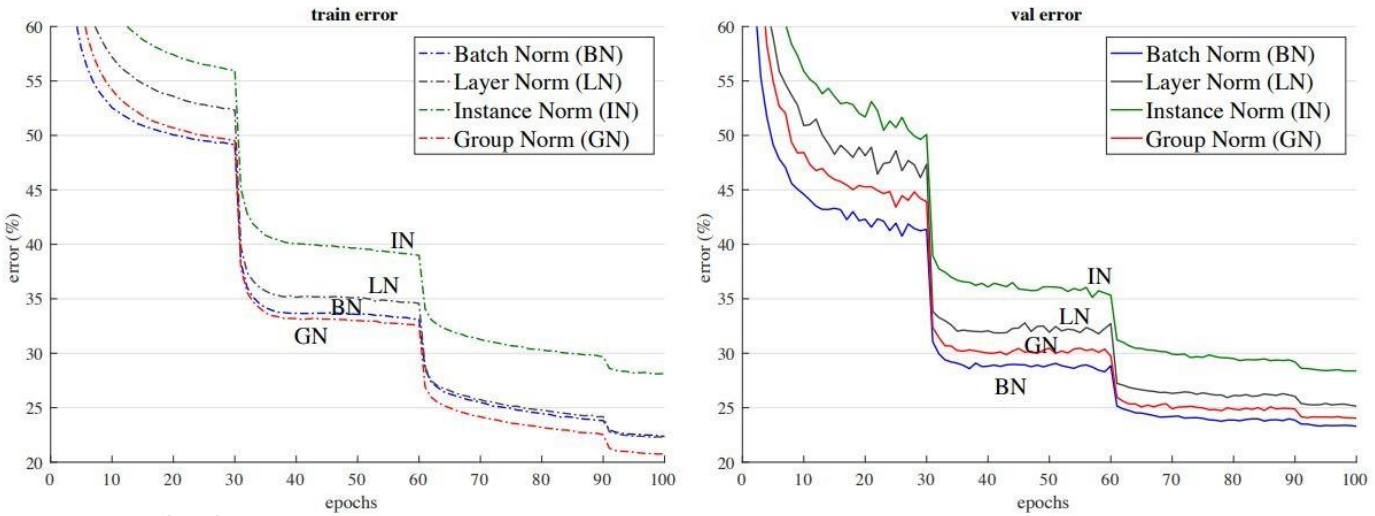


Fig 1: A feature map tensor, with N as the batch axis, C as the channel axis, and (H, W) as the spatial axes. The pixels in blue are normalized by the same mean and variance, computed by aggregating the values of these pixels. [8]

Fig 1: . Comparison of error curves with a batch size of 32 images/GPU for ResNet-50. Given aside is plot of the ImageNet training error (left) and validation error (right) vs. numbers of training epochs. [8]



Optimization algorithms

Optimization algorithms

- Mathematical techniques used to adjust a model's parameters to minimize errors and improve accuracy
- Help models learn from data by finding the best possible solution through iterative updates.

Challenges in Neural Network Optimization

1. Ill-Conditioning-

- Ill-conditioning can manifest by causing SGD to get “stuck” in the sense that even very small steps increase the cost function.
- From the literature if the below mentioned condition is True then we face the challenge of ill-conditioning in convex optimization.

$$(1/2) * learningrate^2 * g^T * H * g > learningrate * g^T * g$$

where g is first order gradient,

H is second order derivative or $Hessian$ matrix

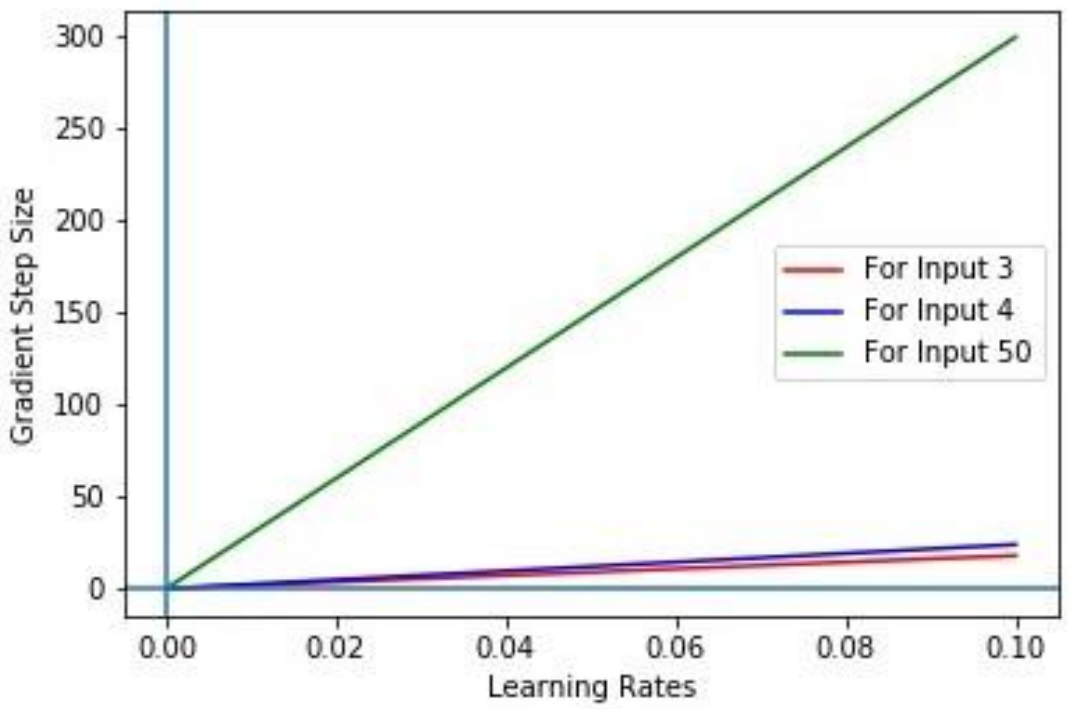
Challenges in Neural Network Optimization

1.Ill-Conditioning-

- Let us consider 3 inputs for x as 3.0, 4.0 and 50.0 and a weight function $f(x) = 30x^2$.
- From the function ,the first order derivative to be $f'(x) = 60x$ and second order derivative to be $f''(x) = 60$.

Challenges in Neural Network Optimization

1.Ill-Conditioning-

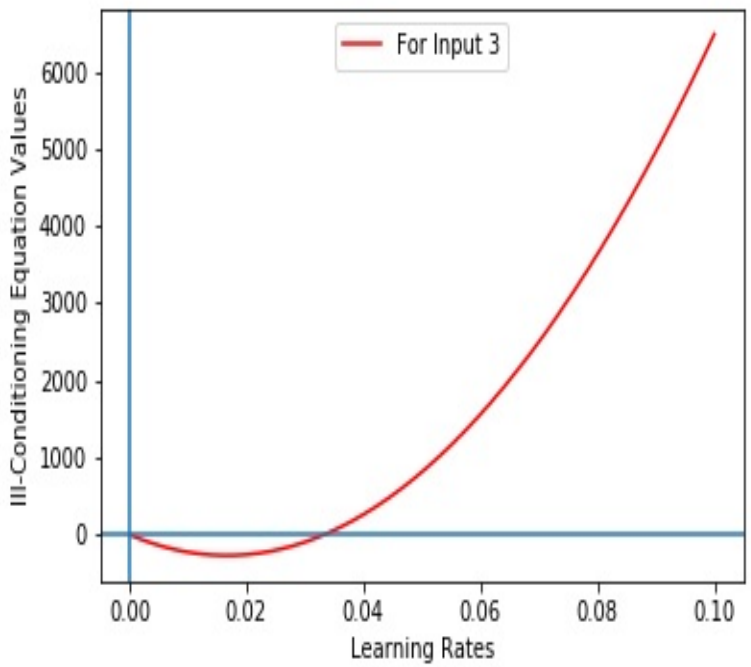


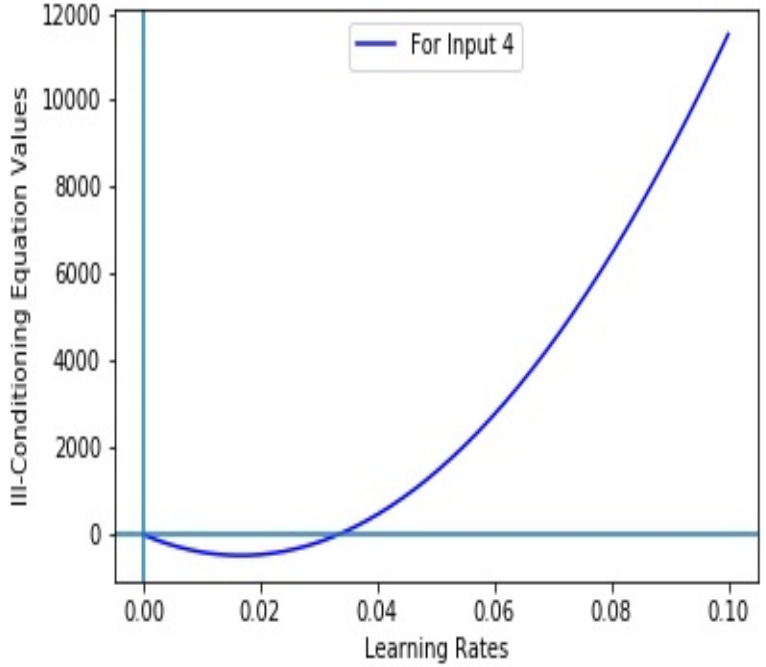
Weight Updates w.r.t Learning Rates

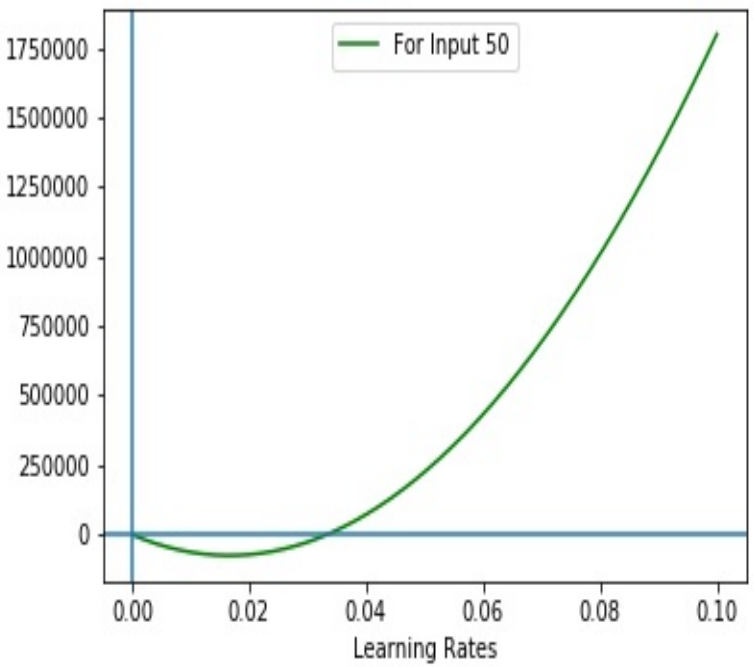
- Larger the size of the input the larger is the gradient step size.
- Find out the approximate learning rate which may lead to ill-conditioning.
- Obtain the values for the ill-conditioning equation and plot it.

Challenges in Neural Network Optimization

1.Ill-Conditioning-







*Ill-Conditioning Equation Values (**if positive it leads to ill-conditioning**)*

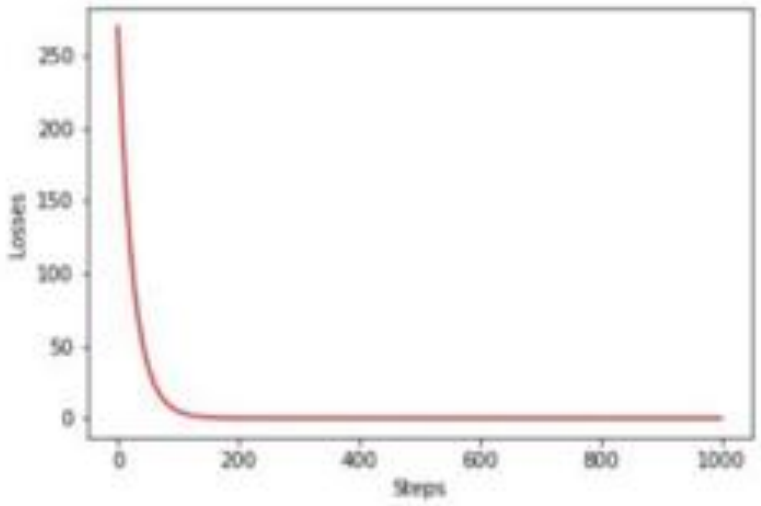
Challenges in Neural Network Optimization

Ill-Conditioning-

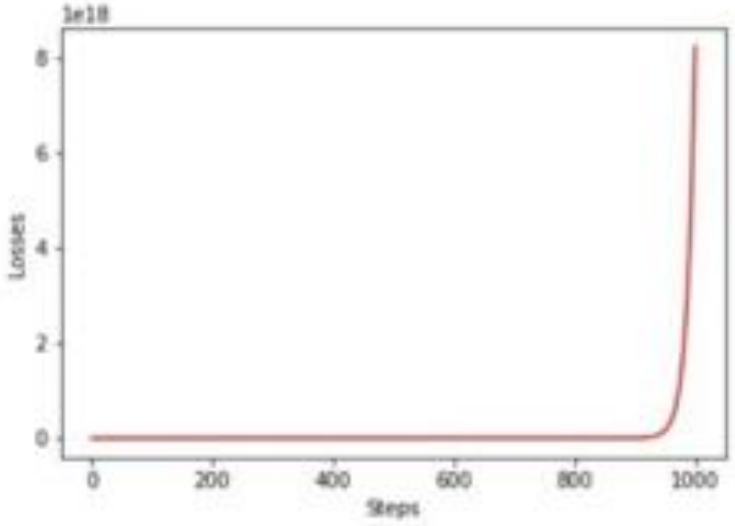
- Proof of Ill-Conditioning-
- To prove ill-conditioning take three scenarios.
  - No ill-conditioning learning rate
  - Learning rates on the border which may cause ill-conditioning if we let it run for a long time
  - Learning rate which will definitely cause ill-conditioning

Challenges in Neural Network Optimization

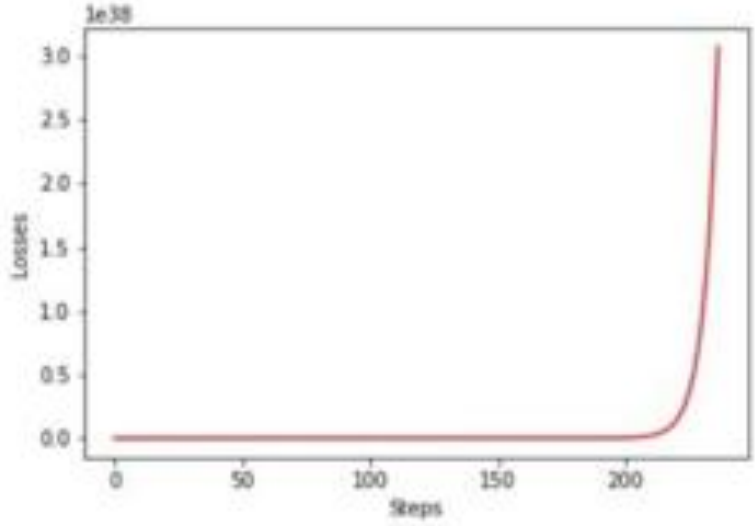
1. Ill-Conditioning-



For Input 3 and Learning Rate 0.033



For Input 3 and Learning Rate 0.034



For Input 3 and Learning Rate 0.035

Challenges in Neural Network Optimization

1. Ill-Conditioning-

Identifiers that there is Ill-conditioning-

- Loss values do not seem to decrease. This could mean that the loss function performs larger updates which leads the gradient to bounce around in the function.
- Sometimes NaN values can be observed in the loss values. One explanation for this is that the loss values could have exploded due to larger learning rates or larger weights. In addition to ill-conditioning, this can also be an indicator for exploding gradients problem.

Challenges in Neural Network Optimization

1. Ill-Conditioning-

How to solve Ill-conditioning

- **Using adaptive learning methods or SGD with annealing-**SGD with a constant learning rate may or may not cause ill-conditioning as the gradient approaches the local minima. Adaptive learning rates like Adam and Adadelta have been observed to perform better than SGD.
- **Pre-processing of the inputs**-Even something simple as pre-processing could be a solution to the ill-conditioning problem.
  - normalization

Challenges in Neural Network Optimization

2. Local Minima

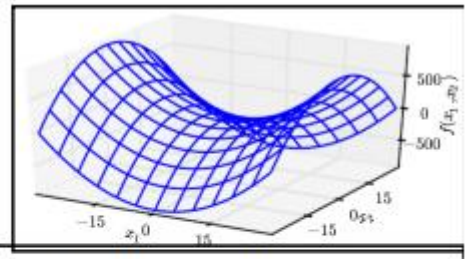
- In convex optimization, problem is one of finding a local minimum
- Some convex functions have a flat region rather than a global minimum point
- Any point within the flat region is acceptable
- With non-convexity of neural nets many local minima are possible
- Many deep models are guaranteed to have an extremely large no. of local minima

Challenges in Neural Network Optimization

3. Plateaus, Saddle Points and Other Flat Regions-

More common than local minima/maxima are:

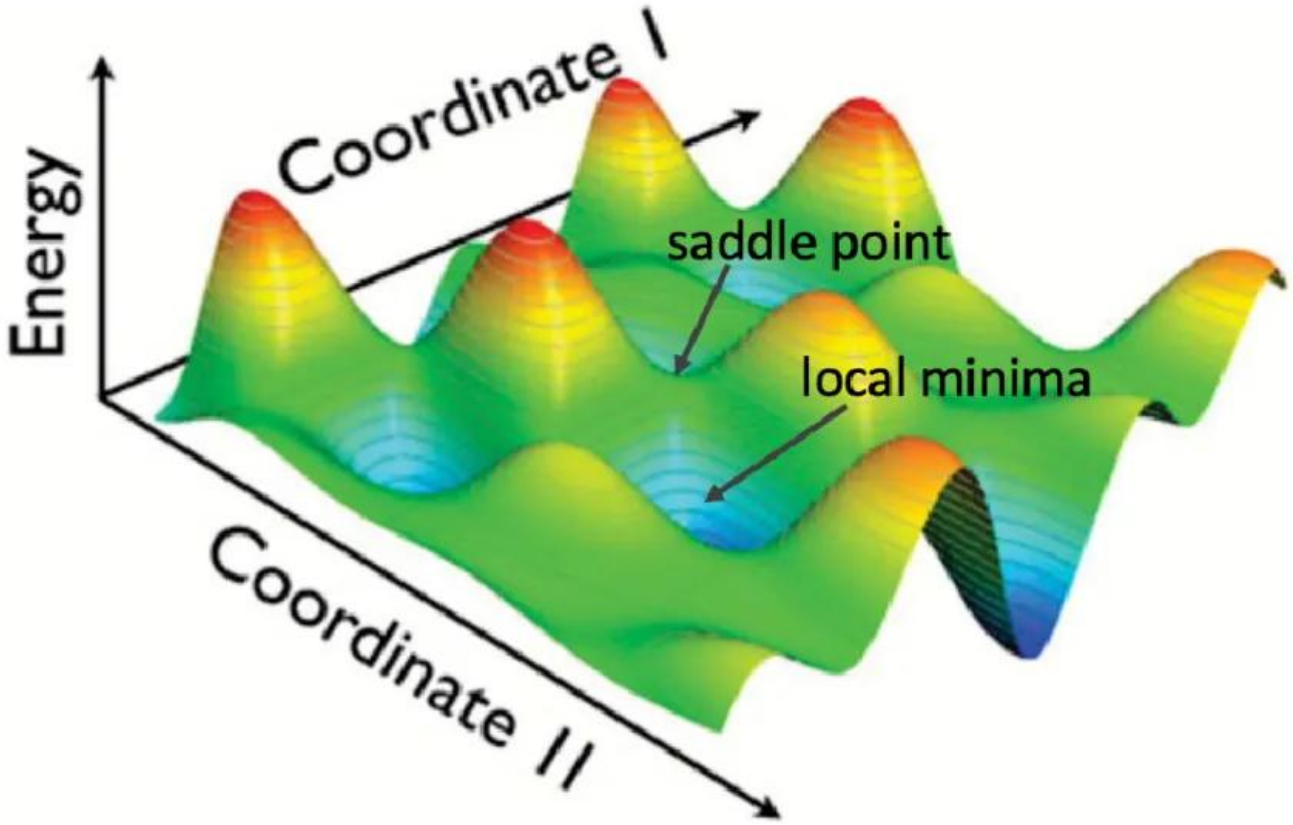
- Another kind of zero gradient points: saddle points
  - At saddle, Hessian has both positive and negative values
    - Positive: cost greater than saddle point
    - Negative values have lower value
- In low dimensions:
  - Local minima are more common
- In high dimensions:
  - Local minima are rare, saddle points more common

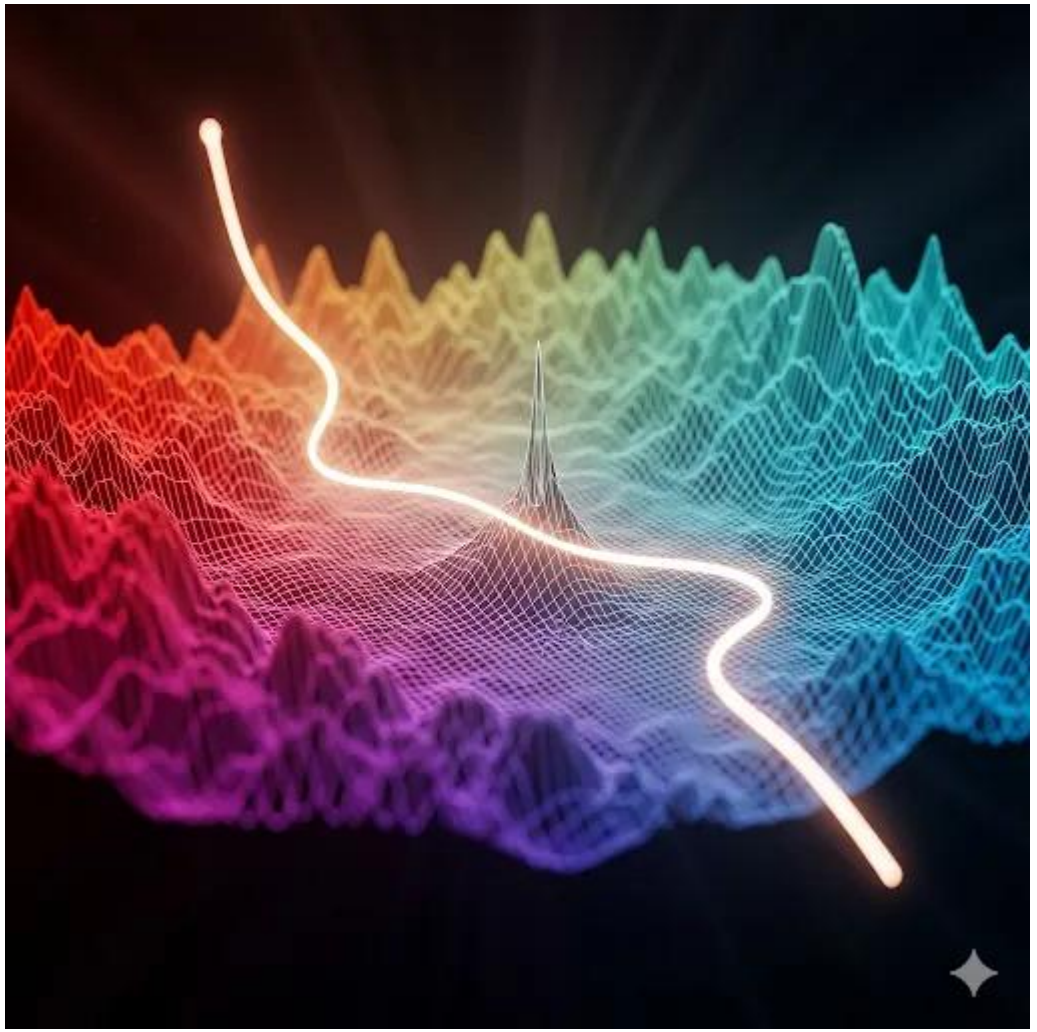


Contains both positive and negative curvature
Function is $f(x)=x_1^2-x_2^2$

Challenges in Neural Network Optimization

Saddle Points

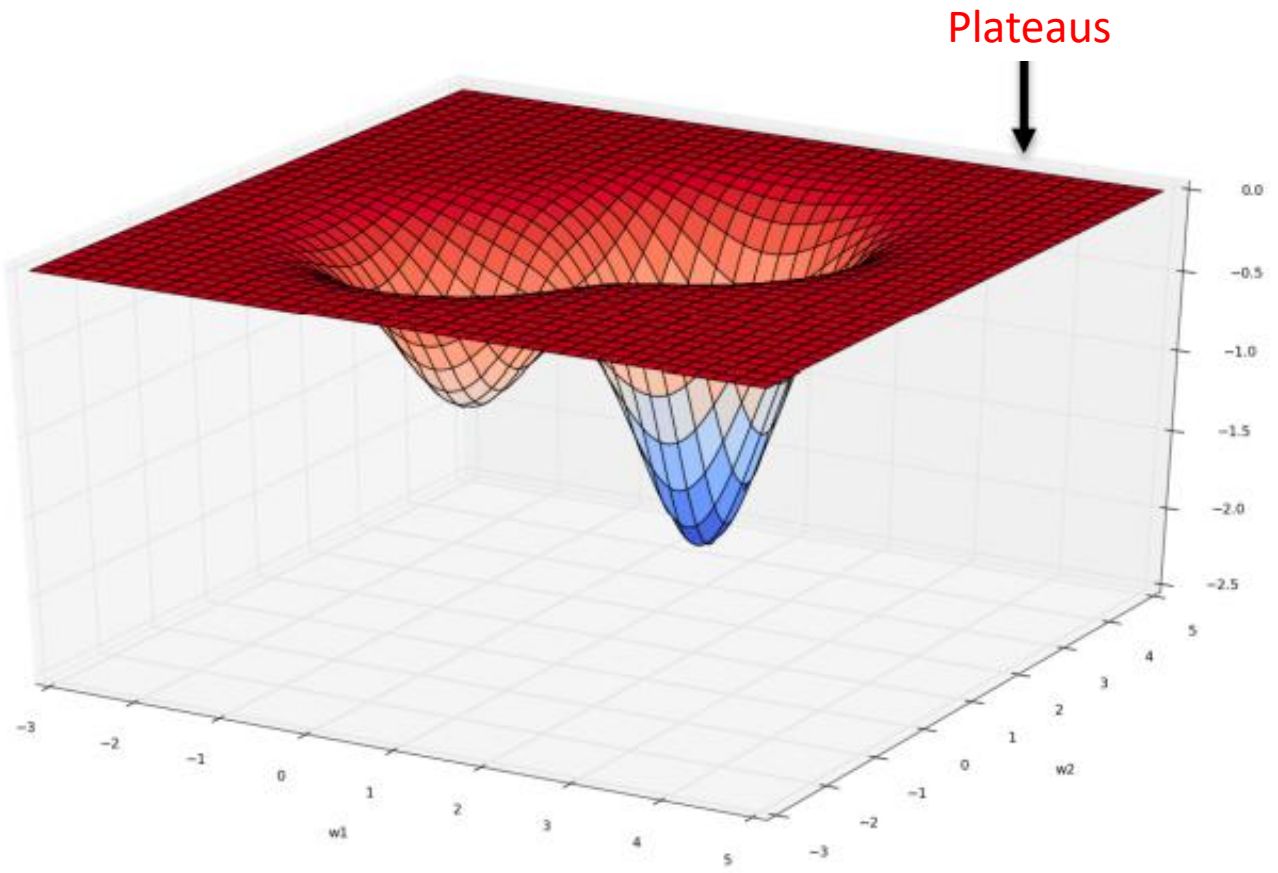




Challenges in Neural Network Optimization

Plateaus

- **Definition:** A state in which the loss value does not decrease significantly, even with further training epochs.
- **Causes:** The primary causes include complex, non-convex loss landscapes, the presence of flat saddle points, the symmetry of neural network models (leading to singular points in parameter space), and inefficient gradient flows, especially in early, under-trained models.
- **Symptoms:** The training loss, after initial rapid decline, stabilizes at a certain value for an extended period, creating a "plateau" in the learning curve.
- **Data Impact:** Data with low covariance (small, dispersed eigenvalues) can make plateaus more pronounced.



Challenges in Neural Network Optimization

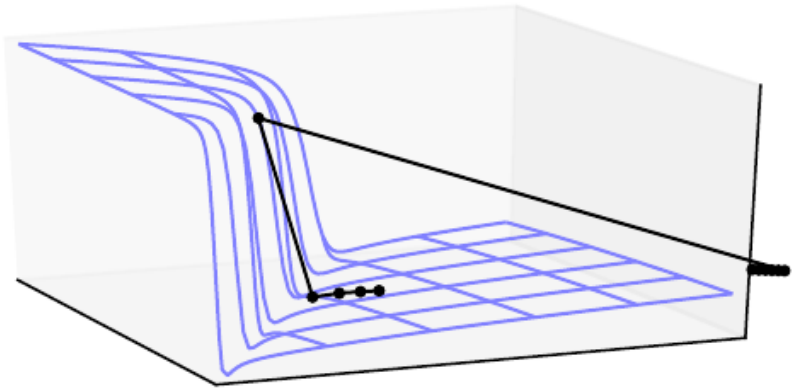
Overcoming Plateaus

- **Learning Rate Adjustment**: Modifying the learning rate, such as using schedulers, reducing it when learning stops, or using optimization algorithms like Adam/RMSprop, can help the model escape plateaus.
- **Activation Patterns**: Algorithms like Active Neuron Least Squares (ANLS) are designed to actively change activation patterns to escape stagnant training.
- **Weight Initialization & Architecture**: Using proper initialization techniques (e.g., Xavier or He initialization) can avoid premature entry into plateau regions.

Challenges in Neural Network Optimization

4. Cliffs and Exploding Gradients-

- Neural networks with many layers often have extremely steep regions resembling cliffs
- These result from the multiplication of several large weights together.
- These are sharp, steep regions in the weight space—often next to flat valleys—where small changes in parameters cause massive jumps in the loss function.
- On the face of an extremely steep cliff structure, the gradient update step can move the parameters extremely far, usually jumping off of the cliff structure altogether.



Optimization algorithms

- Minibatch Gradient Descent,
- Exponentially Weighted Averages,
- Gradient Descent with Momentum,
- AdaGrad,
- RMSProp,
- Adam

Basic Algorithms

- Stochastic Gradient Descent-

Algorithm 8.1 Stochastic gradient descent (SGD) update at training iteration k

Require: Learning rate ϵ_k.

Require: Initial parameter $\boldsymbol{\theta}$

while stopping criterion not met **do**

Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.

Compute gradient estimate: $\hat{\boldsymbol{g}} \leftarrow +\frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$

Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon \hat{\boldsymbol{g}}$

end while

Model update using a single training example at a time which does not require a large amount of computation and therefore is suitable for large datasets.

Basic Algorithms

Stochastic Gradient Descent with Momentum-

- Momentum algorithm introduces a variable *v* that plays the role of velocity—it is the direction and speed at which the parameters move through parameter space.
- The velocity is set to an exponentially decaying average of the negative gradient
- Momentum improves SGD by adding information of the previous steps of the algorithm to the next step.
- By adding a portion of the current update vector to the previous update, it enables the algorithm to go through flat areas and noisy gradients to minimize the time to train and find convergence.

Basic Algorithms

- Stochastic Gradient Descent with Momentum

Algorithm 8.2 Stochastic gradient descent (SGD) with momentum

Require: Learning rate ϵ, momentum parameter α.
Require: Initial parameter $\boldsymbol{\theta}$, initial velocity $\boldsymbol{v}$.
while stopping criterion not met **do**
Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.
Compute gradient estimate: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
Compute velocity update: $\boldsymbol{v} \leftarrow \alpha \boldsymbol{v} - \epsilon \boldsymbol{g}$
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \boldsymbol{v}$
end while

Basic Algorithms

Nesterov Momentum-

- Calculates the gradient at a position slightly ahead in the direction of the accumulated momentum.
- Improves upon standard momentum by calculating gradients at a predicted future position rather than the current one.
- It reduces oscillations, increases training speed, and ensures more stable convergence by incorporating future velocity information.
- This look-ahead step allows the optimizer to correct its course more responsively if it is heading towards a suboptimal direction.

Basic Algorithms

Nesterov Momentum-

- **Advantages:** It provides faster convergence rates, especially in convex problems, and offers better, more consistent, and stable, navigation of the loss landscape.
- **Application:** It is commonly integrated into stochastic gradient descent (SGD) and forms the basis for advanced optimizers like Nadam.

Basic Algorithms

- Nesterov Momentum-

Algorithm 8.3 Stochastic gradient descent (SGD) with Nesterov momentum

Require: Learning rate ϵ, momentum parameter α.
Require: Initial parameter $\boldsymbol{\theta}$, initial velocity $\boldsymbol{v}$.
while stopping criterion not met **do**
Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding labels $\boldsymbol{y}^{(i)}$.
Apply interim update: $\tilde{\boldsymbol{\theta}} \leftarrow \boldsymbol{\theta} + \alpha \boldsymbol{v}$
Compute gradient (at interim point): $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\tilde{\boldsymbol{\theta}}} \sum_i L(f(\boldsymbol{x}^{(i)}; \tilde{\boldsymbol{\theta}}), \boldsymbol{y}^{(i)})$
Compute velocity update: $\boldsymbol{v} \leftarrow \alpha \boldsymbol{v} - \epsilon \boldsymbol{g}$
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \boldsymbol{v}$
end while

Algorithms with Adaptive Learning Rates

1. AdaGrad-

- Optimization method that adjusts the learning rate for each parameter during training.
- Uses past gradients to scale updates making it effective for sparse data and varying feature magnitudes.

Algorithm 8.4 The AdaGrad algorithm

Require: Global learning rate ϵ

Require: Initial parameter $\boldsymbol{\theta}$

Require: Small constant δ, perhaps 10^{-7}, for numerical stability

Initialize gradient accumulation variable $\boldsymbol{r} = \mathbf{0}$

while stopping criterion not met **do**

Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.

Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$

Accumulate squared gradient: $\boldsymbol{r} \leftarrow \boldsymbol{r} + \boldsymbol{g} \odot \boldsymbol{g}$

Compute update: $\Delta\boldsymbol{\theta} \leftarrow -\frac{\epsilon}{\delta + \sqrt{\boldsymbol{r}}} \odot \boldsymbol{g}$. (Division and square root applied element-wise)

Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$

end while

Algorithms with Adaptive Learning Rates

2. RMSProp (Root Mean Square Propagation):

- Addresses the diminishing learning rate issue by introducing an exponentially decaying average of the squared gradients instead of accumulating the sum.
- This prevents the learning rate from decreasing too quickly, making the algorithm more effective in training deep neural networks.

Algorithm 8.5 The RMSProp algorithm

Require: Global learning rate ϵ, decay rate ρ.

Require: Initial parameter $\boldsymbol{\theta}$

Require: Small constant δ, usually 10^{-6}, used to stabilize division by small numbers.

Initialize accumulation variables $\boldsymbol{r} = 0$

while stopping criterion not met **do**

Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.

Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$

Accumulate squared gradient: $\boldsymbol{r} \leftarrow \rho \boldsymbol{r} + (1 - \rho) \boldsymbol{g} \odot \boldsymbol{g}$

Compute parameter update: $\Delta \boldsymbol{\theta} = -\frac{\epsilon}{\sqrt{\delta + \boldsymbol{r}}} \odot \boldsymbol{g}$. ($\frac{1}{\sqrt{\delta + \boldsymbol{r}}}$ applied element-wise)

Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta \boldsymbol{\theta}$

end while

Algorithms with Adaptive Learning Rates

- 2. RMSProp-

Algorithm 8.6 RMSProp algorithm with Nesterov momentum

Require: Global learning rate ϵ, decay rate ρ, momentum coefficient α.

Require: Initial parameter $\boldsymbol{\theta}$, initial velocity $\boldsymbol{v}$.

Initialize accumulation variable $\boldsymbol{r} = 0$

while stopping criterion not met **do**

Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.

Compute interim update: $\tilde{\boldsymbol{\theta}} \leftarrow \boldsymbol{\theta} + \alpha \boldsymbol{v}$

Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\tilde{\boldsymbol{\theta}}} \sum_i L(f(\boldsymbol{x}^{(i)}; \tilde{\boldsymbol{\theta}}), \boldsymbol{y}^{(i)})$

Accumulate gradient: $\boldsymbol{r} \leftarrow \rho \boldsymbol{r} + (1 - \rho) \boldsymbol{g} \odot \boldsymbol{g}$

Compute velocity update: $\boldsymbol{v} \leftarrow \alpha \boldsymbol{v} - \frac{\epsilon}{\sqrt{\boldsymbol{r}}} \odot \boldsymbol{g}$. ($\frac{1}{\sqrt{\boldsymbol{r}}}$ applied element-wise)

Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \boldsymbol{v}$

end while

Algorithms with Adaptive Learning Rates

- Adam-
- Adam (Adaptive Moment Estimation) optimizer combines the advantages of Momentum and RMSprop techniques to adjust learning rates during training.
- Works well with large datasets and complex models because it uses memory efficiently and adapts the learning rate for each parameter automatically.

Algorithms with Adaptive Learning Rates

- **How Does Adam Work?**
- Adam builds upon two key concepts in optimization:
  - **1. Momentum**
    - Used to accelerate the gradient descent process by incorporating an exponentially weighted moving average of past gradients.
    - Smooths out the trajectory of the optimization allowing the algorithm to converge faster by reducing oscillations
  - **2. RMSprop (Root Mean Square Propagation)**
    - uses an exponentially weighted moving average of squared gradients, which helps overcome the problem of diminishing learning rates.

Algorithms with Adaptive Learning Rates

- **Combining Momentum and RMSprop to form Adam Optimizer**
  - Adam optimizer combines the momentum and RMSprop techniques to provide a more balanced and efficient optimization process.

Algorithms with Adaptive Learning Rates

- **Why Adam Works So Well?-**
- **Dynamic learning rates**: Each parameter has its own adaptive learning rate based on past gradients and their magnitudes. This helps the optimizer avoid oscillations and get past local minima more effectively.
- **Bias correction**: By adjusting for the initial bias when the first and second moment estimates are close to zero helping to prevent early-stage instability.
- **Efficient performance**: Adam typically requires fewer hyperparameter tuning adjustments compared to other optimization algorithms like SGD making it a more convenient choice for most problems.

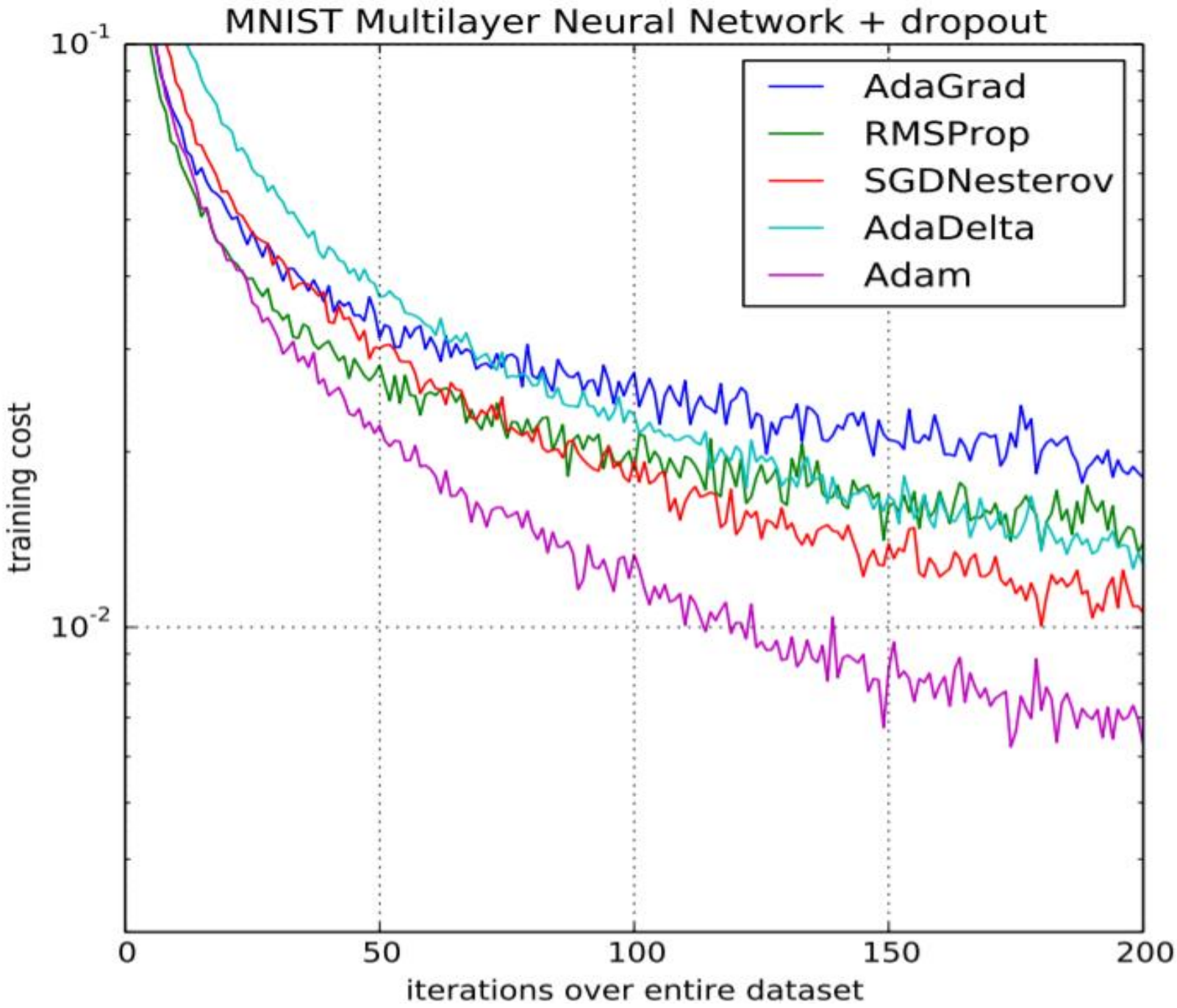
Algorithms with Adaptive Learning Rates

3. Adam

Algorithm 8.7 The Adam algorithm

Require: Step size ϵ (Suggested default: 0.001)
Require: Exponential decay rates for moment estimates, ρ_1 and ρ_2 in $[0, 1)$. (Suggested defaults: 0.9 and 0.999 respectively)
Require: Small constant δ used for numerical stabilization. (Suggested default: 10^{-8})
Require: Initial parameters $\boldsymbol{\theta}$
  Initialize 1st and 2nd moment variables $\boldsymbol{s} = \mathbf{0}$, $\boldsymbol{r} = \mathbf{0}$
  Initialize time step $t = 0$
  while stopping criterion not met **do**
    Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.
    Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
    $t \leftarrow t + 1$
    Update biased first moment estimate: $\boldsymbol{s} \leftarrow \rho_1 \boldsymbol{s} + (1 - \rho_1)\boldsymbol{g}$
    Update biased second moment estimate: $\boldsymbol{r} \leftarrow \rho_2 \boldsymbol{r} + (1 - \rho_2)\boldsymbol{g} \odot \boldsymbol{g}$
    Correct bias in first moment: $\hat{\boldsymbol{s}} \leftarrow \frac{\boldsymbol{s}}{1 - \rho_1^t}$
    Correct bias in second moment: $\hat{\boldsymbol{r}} \leftarrow \frac{\boldsymbol{r}}{1 - \rho_2^t}$
    Compute update: $\Delta\boldsymbol{\theta} = -\epsilon \frac{\hat{\boldsymbol{s}}}{\sqrt{\hat{\boldsymbol{r}}} + \delta}$ (operations applied element-wise)
    Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$
  end while

- Adam


MNIST Multilayer Neural Network + dropout
AdaGrad
RMSProp
SGDNesterov
AdaDelta
Adam
training cost
10^{-1}
10^{-2}
0
50
100
150
200
iterations over entire dataset

Optimization Strategies

Optimizer	Advantages	Disadvantages
SGD	Simple, easy to implement	Slow convergence, requires tuning
Mini-Batch SGD	Faster than SGD	Computationally expensive, stuck in local minima
SGD with Momentum	Faster convergence, reduces noise	Requires careful tuning of β
AdaGrad	Adaptive learning rates	Decays too fast, slow convergence
RMSProp	Prevents fast decay of learning rates	Computationally expensive
Adam	Fast, combines momentum and RMSProp	Memory-intensive, computationally expensive

Exponentially Weighted Averages

- Smoothing technique used to calculate moving averages by giving higher weight to recent data points while exponentially decreasing the weight of older data, improving optimization stability.
- Crucial component in optimizers like Momentum, RMSprop, and Adam, providing faster convergence by reducing oscillations.

Exponentially Weighted Averages

Key Concepts of EWA in Deep Learning

- **Formula:** $V_t = \beta V_{t-1} + (1 - \beta)\theta_t$, where V_t is the moving average at time t, θ_t is the current data point, and β (typically 0.9 to 0.999) determines the weight of the past vs. present.
- **Smoothing Parameter (β):** A higher β (e.g., 0.98) gives more weight to the past, resulting in a smoother curve, while a lower β (e.g., 0.5) makes the average more sensitive to recent changes.

Exponentially Weighted Averages

- **Applications:**
  - **Momentum:** Uses EWA to compute a running average of gradients, reducing noise and oscillation to accelerate training.
  - **RMSprop & Adam:** Utilize EWA to track the squared gradients (moving variance) for adaptive learning rates, preventing overly large steps.

Exponentially Weighted Averages

- **Benefits of EWA**
  - **Reduces Noise**: Smoothes out the training cost/gradient, making convergence, particularly in mini-batch gradient descent, more stable.
  - **Handles Data Fluctuations**: Allows optimizers to adapt to changing landscapes in loss functions.

References

1 Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift, arXiv preprint arXiv:1502.03167.

2 Santurkar, S., Tsipras, D., Ilyas, A., & Madry, A. (2018). How does batch normalization help optimization?, Advances in Neural Information Processing Systems .

3 Johann Huber(2020), Batch normalization in 3 levels of understanding, Towards Data Science.

4 Bala Priya C, Build Better Deep Learning Models with Batch and Layer Normalization, pinecone.io

5 Ian Goodfellow, Ch 9: Convolutional Networks, Youtube.

6 Jimmy Lei Ba, Jamie Ryan Kiros, Geoffrey E. Hinton(2016), Layer Normalization, arXiv preprint arXiv:1607.06450.

7 Normalization, paperswithcode.com

8 Yuxin Wu, Kaiming He (2018), Group Normalization, arXiv:1803.08494v3

9 Elad Hoffer, , Itay Hubara, , Daniel Soudry (2018) ,Train longer, generalize better: closing the generalization gap in large batch training of neural networks, arXiv:1705.08741v2

10 Cecilia Summers, Michael J. Dinneen (2020), Four Things Everyone should know to improve Batch Normalization, arXiv:1906.03548v2

References

- Ian Goodfellow, Yoshua Bengio, Aaron Courville, “Deep Learning”, MIT Press 2017
- Josh Patterson and Adam Gibson, “Deep Learning A Practitioner’s Approach”, O’Reilly Media, 2017
- https://towardsdatascience.com/regularization-an-important-concept-in-machine-learning-5891628907ea
- https://www.analyticsvidhya.com/blog/2021/05/complete-guide-to-regularization-techniques-in-machine-learning/
- https://www.geeksforgeeks.org/regularization-in-machine-learning/